# Financing with Doug

## User's Manual and Calculation Guide

Accounts, budgets, goals, and forecasts



*Doug's self-portrait. Accuracy not independently verified.*

Dr. Caleb N. Harper

Doug 1.22.0 ▪ Manual revision 1.9

Prepared October 17, 2026

# About this manual

Doug records financial information supplied by its user and uses that information to describe history and estimate future outcomes. The purpose of this manual is to explain how to perform those tasks, what each result means, and which records or assumptions should be checked when the result is unexpected. Prior knowledge of accounting, statistics, or the application is not required.

The first part provides a starting path and the conventions used throughout the application. The task chapters then explain individual activities, including importing statements, locating goal savings, building a monthly budget, and comparing forecasts. Worked examples connect these activities with amounts that can be checked independently. The calculation chapters develop the underlying methods for readers who want to understand why the application produces a particular result. Appendix D describes how to keep the manual aligned with the implementation.

**Scope of this edition**

This edition describes Doug 1.22.0 and the working implementation inspected on 2026-10-17. GUI review was in progress during preparation. It follows the grouped navigation shown in Chapter 3; older installed releases may place the same pages directly in the sidebar. A source commit alone does not identify uncommitted GUI work. The recorded example manifest therefore includes the SHA-256 hashes of the inspected application files.

The text uses dollars because that is the currency convention implemented by the inspected application. It does not describe currency conversion or independent ledgers in multiple currencies. Figures and worked records are fictional. The diagrams illustrate relationships without depending on the exact position or color of a control in a transient GUI layout.

## How to use the book

Read Chapters 2–3 for a first session. For an immediate task, use the table below or the linked contents. A task chapter explains its purpose, the procedure, the expected result, and the checks to make when the result differs. The detailed calculation chapters can be read separately.

| I want to… | Start here |
|---|---|
| Set up accounts and import a statement | Chapters 4 and 5 |
| Understand a balance difference | §4.4, p. 11; Appendix B |
| Correct a category or a reimbursement | Chapter 6 |
| See upcoming bills and paydays | Chapter 7 |
| See which accounts hold my goal savings | Chapter 9 |
| Make room in a monthly budget | Chapter 10 |
| Test a change in spending or income | Chapter 11 |
| Compare a saved forecast with later records | Chapter 12 |
| Back up or restore a database | Chapter 13 |
| Check a formula or maintain this document | Part IV and Appendix D |

## Interpretation of results

A forecast is conditional on the records, assumptions, and model used to construct it. Missing transactions, changed income, unrecorded commitments, unusual expenses, and different investment returns can produce outcomes outside the displayed bands. A data-quality check identifies records to inspect; passing it does not establish completeness or forecast accuracy. A spending score is a benchmark calculation rather than an assessment of a person's circumstances. The manual explains the program's behavior and does not prescribe a financial decision.

## Revision history

| Revision | Date | Description |
|---|---|---|
| 1.0 | 2026-10-17 | First standalone edition: setup, task workflows, account and goal examples, calculation methods, troubleshooting, and reproducible maintenance instructions. Application integration is deferred. |
| 1.1 | 2026-10-17 | Illustrated textbook cover reflecting Doug's family backstory. Calculation evidence and application baseline are retained. |
| 1.2 | 2026-10-17 | Refined cover with a technical illustration, restrained palette, and serif title typography. |
| 1.3 | 2026-10-17 | Added Doug's buck-toothed grin, pen-filled shirt pocket, and bow tie while retaining the textbook illustration style. |
| 1.4 | 2026-10-17 | Revised Doug with curly hair, lankier proportions, and an awkward deadpan expression. |
| 1.5 | 2026-10-17 | Added exaggerated round glasses and thick magnifying lenses using the supplied expression reference. |
| 1.6 | 2026-10-17 | Replaced the cover with a fresh full-color portrait based directly on the supplied photograph. |
| 1.7 | 2026-10-17 | Centered title-page layout, portrait caption, and clearer author and edition details. |
| 1.8 | 2026-10-17 | Added Help > User manual (PDF), bundled-file opening, and packaging checks. Calculation evidence is retained. |
| 1.9 | 2026-10-17 | Updated for Doug 1.22.0: View > Appearance (light and dark), deleting a goal activity, first-use panels, Home's new order and Needs a look, the Forecast panel in Simple view, What Doug read on import, the calendar's today ring, keyboard behavior, and the date convention. Examples regenerated. |

The examples were generated against Doug 1.22.0, commit `25b025e`, with local modifications recorded in the provenance manifest. Their generation date is 2026-10-17. A PDF-only build reuses those retained calculations. It does not rerun the application or turn an earlier observation into a fresh verification result.

# Contents

**About this manual** i

**I Start here** 1

**1 What Doug records and estimates** 2

1.1 The questions the application answers . . . 2
1.2 A useful working order . . . 2
1.3 The meaning of local data . . . 3
1.4 What the application does not establish . . . 3

**2 Installation and a first session** 4

2.1 Install the Windows application . . . 4
2.2 Begin with fictional data . . . 4
2.3 Set up an actual ledger . . . 4
2.4 Run from source . . . 5
2.5 Know where the records are stored . . . 5

**3 Navigation and conventions** 6

3.1 Find a page by its task . . . 6
3.2 Enter money deliberately . . . 6
3.3 Signs, balances, and dates . . . 7
3.4 Selection, scrolling, and charts . . . 7
3.5 Stored changes and temporary changes . . . 8

**II Using Doug** 9

**4 Accounts and balance reconciliation** 10

4.1 Choose the account type . . . 10
4.2 Use an observation you can verify . . . 10
4.3 Update transactions and balances separately . . . 10
4.4 Investigate a difference . . . 11

4.5 Read running balances carefully . . . 11
4.6 Edit or remove an account . . . 12

**5 Importing statements 13**
5.1 Prepare the export . . . 13
5.2 Map the columns . . . 13
5.3 Import and inspect the result . . . 14
5.4 Overlapping exports and identical purchases . . . 14
5.5 Correct earlier date mappings . . . 14
5.6 Pending entries and malformed rows . . . 14
5.7 Loan and investment imports . . . 15

**6 Reviewing transactions and categories 16**
6.1 Review the evidence behind a category . . . 16
6.2 Keep one-off decisions separate from merchant rules . . . 16
6.3 Review merchants in groups . . . 17
6.4 Recognize internal transfers . . . 17
6.5 Separate reimbursable business activity . . . 18
6.6 Export reviewed data deliberately . . . 18

**7 Bills, paydays, and the calendar 19**
7.1 Review a detected schedule . . . 19
7.2 Link a loan payment . . . 19
7.3 Read the calendar as a schedule . . . 19
7.4 Check the payment account . . . 20
7.5 When circumstances change . . . 20

**8 Reading Home and spending 21**
8.1 Begin with the balances and their evidence . . . 21
8.2 Inspect monthly amounts before the grade . . . 21
8.3 Interpret the score as a benchmark . . . 22
8.4 Trace a category to its ledger . . . 22
8.5 Turn an observation into an explicit plan . . . 22

**9 Goals and the location of savings 23**
9.1 Choose what the goal measures . . . 23

9.2 Create a savings goal . . . 23
9.3 Read the funding map . . . 23
9.4 Record what actually happened . . . 24
9.5 Understand the funding checks . . . 25
9.6 Set the monthly priority . . . 25
9.7 Delete a goal deliberately . . . 25

**10 Building the monthly budget** **26**
10.1 Separate the available amount from the chosen allowance . . . 26
10.2 Build a plan from recorded evidence . . . 26
10.3 Adjust categories without counting them twice . . . 26
10.4 Use the shared allowance . . . 27
10.5 Connect the plan with Forecast . . . 27

**11 Forecasts and scenarios** **28**
11.1 Establish the baseline before changing it . . . 28
11.2 Read the curve and bands . . . 28
11.3 Check data quality as evidence . . . 29
11.4 Compare one explicit change . . . 29
11.5 Use the saved budget deliberately . . . 29
11.6 Add a hypothetical expense or income . . . 30
11.7 Select a relevant historical period . . . 30
11.8 Investigate a modeled cash dip . . . 31

**12 Track record and reports** **32**
12.1 Save the claim before the outcome is known . . . 32
12.2 Compare an earlier forecast with later balances . . . 32
12.3 Distinguish saved and reconstructed checkpoints . . . 32
12.4 Interpret a spending-change result . . . 32
12.5 Export a month in review . . . 33
12.6 Maintain useful comparisons . . . 33

**13 Backups, updates, and recovery** **34**
13.1 Back up the database that is actually open . . . 34
13.2 Restore without overwriting the only copy . . . 34
13.3 Use Undo within a session . . . 34

13.4 Update an installed application . . . 35
13.5 Recover from an unexpected result . . . 35

III Worked examples 36

14 A worked household example 37
14.1 Check a dated balance . . . 37
14.2 Locate four thousand dollars . . . 37
14.3 Allocate a monthly allowance . . . 38
14.4 Make room in the budget . . . 39
14.5 Compare the conditional forecast . . . 40
14.6 Repeat the examples . . . 40

IV How the calculations work 42

15 Ledger and spending calculations 43
15.1 Reconstruct a cash ledger around an observation . . . 43
15.2 Combine account histories . . . 43
15.3 Separate personal spending from ledger flow . . . 43
15.4 Estimate a typical month . . . 44
15.5 Calculate the spending score . . . 44

16 Goal allocation and the budget model 46
16.1 Keep holdings, claims, and progress distinct . . . 46
16.2 Build the monthly room . . . 46
16.3 Divide the extra pool . . . 46
16.4 Pay loans and release their commitments . . . 47
16.5 Interpret completion dates . . . 47

17 Forecast construction 49
17.1 Construct a daily path . . . 49
17.2 Replay recurring items without sampling them twice . . . 49
17.3 Sample complete behavioral weeks . . . 50
17.4 Weight a sustained change and retain uncertainty in the mean . . . 50
17.5 Apply a saved monthly plan . . . 50

17.6 Model investments at monthly boundaries . . . 51
17.7 Summarize the paths . . . 51

**18 Evidence and interpretation 52**
18.1 Distinguish four kinds of evidence . . . 52
18.2 Retain forecasts before evaluating them . . . 52
18.3 Interpret coverage descriptively . . . 52
18.4 Compare weekly spending before and after . . . 52
18.5 Understand what the manual checks . . . 53

**A Quick reference 54**
A.1 Minimum records for a useful first forecast . . . 54
A.2 Canonical signed transaction example . . . 54
A.3 Signs in forms and stored records . . . 55
A.4 Actions that are easy to confuse . . . 55
A.5 Keyboard and location reference . . . 55

**B Troubleshooting 56**
B.1 Accounts and imports . . . 56
B.2 Goals and budgets . . . 56
B.3 Forecasts and application state . . . 57

**C Glossary and symbols 59**
C.1 Symbols used in the calculation chapters . . . 60

**D Maintaining the manual 61**
D.1 Keep a task-based source . . . 61
D.2 Use the source map . . . 61
D.3 Build and check an edition . . . 61
D.4 Review what the checks cannot establish . . . 62
D.5 Record revisions and preserve reproducibility . . . 62
D.6 Bundle the matching edition . . . 63

# List of Figures

14.1 One set of holdings, divided among purposes. Account bars share a dollar scale. The $10,000 House goal is 15% funded even though its $1,500 occupies more account space than the fully funded Baseline. . . . . 38

14.2 The intended planning workflow. The allowance is shared between Goals and Monthly budget. Forecast uses the saved plan only when the corresponding option is enabled. . 39

14.3 Fictional cash projection. The light and darker bands are baseline P10–P90 and P25–P75 at each date. Lines show the baseline and saved-plan P50. These conditional model outputs are not measured prediction accuracy. . . . . 40

# Part I

# Start here

Chapter 1

# What Doug records and estimates

## 1.1 The questions the application answers

Financing with Doug is a desktop application for reviewing account balances, imported transactions, spending, goals, and financial projections. Bank information enters through CSV exports or values entered by the user. Doug does not connect to a bank or initiate a payment, transfer, investment sale, or purchase.

The application separates three kinds of information. *Records* describe what has happened: a dated balance, a posted transaction, a purchase, or savings already set aside. *Plans* describe intended monthly amounts and priorities. *Scenarios* change assumptions in a projection so that their modeled consequences can be compared. Keeping these categories separate is necessary because entering a plan does not make the planned action occur.

| Information | Example | Effect |
|---|---|---|
| Recorded balance | Checking held $1,000 at the end of a stated date | Anchors the reconstructed account balance. |
| Recorded goal activity | $300 already set aside for a computer | Records a purpose for existing holdings. |
| Monthly budget | Reduce dining by $100/month | Supplies a saved planning assumption. |
| Temporary scenario | Increase income by 5% | Changes the current forecast comparison. |
| Saved what-if event | A $700 purchase next month | Adds a hypothetical dated cash flow until disabled or removed. |

## 1.2 A useful working order

The account ledger is the starting point for the rest of the application. Enter dated balances, import transactions, and review their categories. Check recurring bills and income before interpreting the calendar or forecast. Then build goals and a budget from those records. This order makes an unexpected projection easier to trace: a changed assumption can be distinguished from a missing bill or a reversed amount sign.

1. Establish the balances and dates you can verify against statements.
2. Import the corresponding transaction history and check the preview.
3. Review uncertain categories, internal transfers, reimbursements, and recurring patterns.
4. Record the savings already assigned to goals and where those savings are held.
5. Build a monthly budget and choose an allowance for goals and extra debt payments.

6. Compare forecasts, then save checkpoints and review them as new records arrive.

## 1.3 The meaning of local data

Doug's working data is stored in a local SQLite database. The status bar identifies the file currently open. A demonstration dataset uses a separate database and is marked by a banner. A backup copies the open database, so a backup made while a demonstration is open contains demonstration data.

Local storage does not by itself provide encryption, an off-device backup, or protection against someone who can read files in the user account. Exported reports and reviewed-data files also contain financial information. The updater contacts GitHub to check for releases and download software; those requests do not include the financial database.

## 1.4 What the application does not establish

An internally consistent ledger can still be incomplete. A balance that matches one statement does not establish that an earlier export contains every transaction. A detected recurring item is an estimate from repeated records, rather than confirmation from a payee. A goal earmark records an intended purpose, rather than a restriction imposed by a bank. Investment holdings do not become immediately withdrawable cash because they appear in the goal map.

The forecast uses a specific representation of scheduled payments, sampled spending, debt, investments, and hypothetical events. It cannot identify expenses or changes that are absent from those inputs. Consequently, its most useful application is comparison: inspect how a result changes when an explicit assumption changes, and check the records supporting that assumption.

Implementation reference:
`finfore/models.py;finfore/store.py;finfore/forecast.py;finfore/update.py`

Chapter 2

# Installation and a first session

## 2.1 Install the Windows application

The Windows distribution is an installer named `FinancingWithDoug-<version>-Setup.exe`. Obtain it from the project's release page at https://github.com/harpe119/Financing-With-Doug/releases/latest, check that the source is the intended project, and follow the installer. This address identifies the distribution location; the manual does not certify which release is currently published there.

An unsigned installer may produce a Windows SmartScreen warning. The warning is not a determination that the file is safe or unsafe. Check the source and the release information before deciding whether to continue. The application identifies its version in the header and under **Help > About Doug**. Use **Check for updates** to ask whether a newer packaged release is available.

## 2.2 Begin with fictional data

For a first examination of the controls, choose **File > Load demo dataset**. The available demonstrations describe different financial situations: Balanced, Struggling, Thriving, Turnaround, and Spiralling. Their purpose is to make application behavior visible without changing the user's financial records.

1. Load a demonstration and confirm that its banner is visible.
2. Open Home, Accounts, Plan, and Forecast. Select a goal and compare its saved amount with its account sources.
3. Make a small temporary forecast adjustment, then use **Reset scenario**.
4. Choose **Return to my own data** on the banner, or **File > Return to my own data**, and confirm that the banner disappears and the status-bar path identifies the intended database.

Demonstration results do not measure prediction accuracy for a real household. Their records were constructed to illustrate controls and particular model behaviors.

## 2.3 Set up an actual ledger

The minimum starting information is an account, a balance, and the date of that balance. Transaction history then explains the changes around the dated observation.

An empty database opens on Home's welcome, which lists three steps (add accounts, import a statement, see where the money is headed) and offers the next one as a button. A page that fills in from statements shows a short panel in place of an empty chart or table: what the page will hold, what it needs, and one button for the next step. Until statements are imported, **Bills & payday** and **Forecast** appear

muted and in italics in the sidebar. They remain available, and each page states what it is waiting for. Choosing **Import CSV** before any account exists opens the account form first.

A useful first session is:

1. Add the accounts described in Chapter 4. Use the posted closing balance for a stated date.
2. Save a backup before importing a large history or correcting an existing ledger.
3. Import one account's CSV and inspect its signs, dates, and descriptions.
4. Review the first and last imported dates and several individual entries against the bank export.
5. Review categories and recurring patterns. Open Forecast only after checking the records on which it depends.

Several complete months are more informative than a few days. The forecast requires at least four usable complete weeks before it includes sampled variable spending. An application that opens successfully with less history has not thereby obtained enough evidence to estimate that spending.

## 2.4 Run from source

This route is intended for users maintaining or examining the program. Python 3.10 or newer and the packages in `requirements.txt` are required. From the repository root:

```
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
pip install -r requirements.txt
python run.py
```

Use an explicit database when examining a copy or experimenting:

```
python run.py --db "path/to/working-copy.db"
```

The file is created if it does not exist. Verify the status-bar path before entering financial records. Launching a source checkout does not update a separately installed Windows application, and changing a source file does not publish an installer.

## 2.5 Know where the records are stored

The default database is `~/.ledgerline/ledgerline.db`; on Windows this corresponds to `C:\Users\<you>\.ledgerline\ledgerline.db`. The internal folder retains the earlier name *Ledgerline*. It is the location shown by the application, rather than the folder name alone, that identifies the current file.

Keep a regular backup of this database. The Undo stack is kept in memory and is not a replacement for a backup. Closing the application ends that undo history. Procedures for backup and restoration are given in Chapter 13.

Implementation reference: `run.py;requirements.txt;finfore/ui/main_window.py;finfore/demo.py`

Chapter 3

# Navigation and conventions

## 3.1 Find a page by its task

The inspected interface groups related pages into five sidebar sections. A path such as **Plan** > **Goals** names a section and then a tab; it does not imply that a second application window opens.

| Section | Pages | Main purpose |
|---|---|---|
| Home | Overview; Spending | Review balances, outlook, and spending. |
| Accounts | Balances; Transactions | Maintain accounts and inspect their ledgers. |
| Bills & payday | Calendar; Detected bills & pay | Review schedules and their account assignments. |
| Plan | Goals; Monthly budget | Set goals, order priorities, and allocate monthly amounts. |
| Forecast | Projection; Track record | Compare assumptions and examine earlier forecasts. |

Older editions may show Dashboard, Goals, Plan, or Progress as separate sidebar entries. In this manual, Home corresponds to the overview; Monthly budget corresponds to the spending-plan editor; Track record corresponds to saved comparisons. The task and tab names provide the route when a layout differs.

The header's **View** control selects Simple or Everything. Simple hides advanced controls and some tabs without deleting their data or saved settings. A link to a hidden page can reveal it. If a control described later is absent, first check the selected view and the current tab.

The same choice is under **View** in the menu bar, which also holds **View** > **Appearance**: Dark (the default), Light, or Match my computer, which follows the operating system's light or dark setting and changes with it. A new appearance applies at once and changes only the colors on screen; the current page, an open panel, typed text, and the scroll position are retained, and the choice is remembered. Each color keeps the same meaning in both appearances, so a description in this manual applies to either.

Choose **Help** > **User manual (PDF)** to open the complete manual in the computer's PDF reader. It opens separately so that it can remain beside the application while following a task. F1 opens the shorter Getting started guide. If Doug cannot launch a PDF reader, its message gives the manual's file location for opening directly.

## 3.2 Enter money deliberately

Currency controls use fixed-decimal entry: digits enter cents from the right. The displayed value is the amount that will be stored, so inspect it before accepting a dialog.

| Digits | Displayed amount |
|---|---|
| `1` | $0.01 |
| `1234` | $12.34 |
| `123400` | $1,234.00 |

Clicking into a money field selects its existing amount. Backspace removes a digit; Delete clears the value. A pasted formatted amount such as `$1,234.56` retains its dollar value, whereas pasted bare digits follow fixed-decimal entry. Changes are committed when the field accepts them, normally on Enter or when focus leaves it. Percentages and relative shares use ordinary decimal entry; they are not entered as cents.

The scroll wheel is reserved for navigation rather than changing entry values. Use the keyboard, explicit step buttons, or a selection to alter a field. A dropdown list that has been opened remains a list through which the user can navigate.

## 3.3 Signs, balances, and dates

Posted transaction amounts are positive for money entering an account and negative for money leaving it. The account form accepts a loan or card amount owed as a positive number. Internally, a card debt is represented by a negative ledger balance, while a loan stores a positive amount owed. A negative card balance in the transaction view therefore means debt.

A dated balance is an observation at the end of that date. A transaction's posting or balance date identifies when it belongs to the ledger. The purchase date can differ, particularly across weekends or holidays. Use the bank's supplied posting date when available; the program does not infer a bank's posting rules from the day of the week.

Dates on screen are written as *Oct 20, 2026*, or as *Tue, Oct 20* where the weekday matters and the year is evident.

## 3.4 Selection, scrolling, and charts

Select an entry before using an action that applies to a row. In Goals, selecting a savings goal in the funding map also selects it in the priority ladder. Selecting a loan or balance milestone retains that target in the ladder and clears the savings highlight, since those items do not represent a separate savings-account allocation.

Charts can be panned or zoomed for inspection. Routine changes to layers or graph contents preserve the current view. **Reset axes** restores the current default extent; **Reset scenario** changes forecast controls instead. These commands have different purposes.

Use Tab and Shift+Tab to move through controls and Escape to leave a dialog without accepting its change. Within a table or list, the arrow keys move between entries and Tab moves on to the next control. The control that has the keyboard shows a visible outline. Charts are read with the pointer (rest on a chart for exact values, scroll to zoom, double-click to reset) and are not Tab stops; the sentence beside a chart states its main result in words. Native keyboard operation does not by itself establish compatibility with every screen reader or display-scaling setting.

## 3.5 Stored changes and temporary changes

Account edits, imported transactions, goals, recorded activity, recurring settings, saved events, and saved budgets affect the database. A temporary forecast multiplier changes an assumption in the current scenario. Saved what-if events remain until managed explicitly. View preferences can be stored separately from the financial database. Before interpreting a result, identify which of these kinds of change was made.

Implementation reference: `finfore/ui/main_window.py;finfore/ui/manual.py;finfore/ui/widgets.py; finfore/ui/input_behavior.py;finfore/ui/charts.py`

# Part II

# Using Doug

Chapter 4

# Accounts and balance reconciliation

## 4.1 Choose the account type

Open **Accounts** > **Balances** and choose **Add account**. Enter a recognizable name, its type, a balance, and the observation date. Account type determines how records are interpreted; it is more than a display label.

| Type | Enter | Review |
|---|---|---|
| Checking or savings | Posted closing balance and date | Ledger completeness; whether negative closes are permitted. |
| Credit card | Amount owed, normally positive in the form | Posting dates, payment transfers, and the bank's sign convention. |
| Loan | Amount owed, APR, regular payment, payment day | Current terms and linked payment patterns. |
| Investment | Account value and date; return, volatility, contribution assumptions | Valuation history and whether assumptions suit the holding. |

A money market account entered as Savings follows the cash-ledger treatment. A CD entered as Investment follows the investment-value treatment. Its balance can be located within a goal, but Doug does not infer its maturity date, withdrawal penalty, or whether additional contributions are allowed. Review those restrictions outside the model before interpreting the holding as available money.

## 4.2 Use an observation you can verify

For a cash ledger, enter the posted balance at the end of the selected date. An available balance may already reflect pending holds and can therefore disagree with a ledger containing only posted transactions. If today's posting is unfinished, use the most recent completed date and its closing balance instead.

The observation anchors reconstruction: Doug moves backward or forward through the ledger around that amount. An incorrect observation date shifts which flows are included at the anchor. Entering a new balance changes the reference; it does not create an explanatory transaction for a discrepancy.

For checking and savings that cannot go below zero, leave **Allow this account to go below $0** unchecked. A flagged negative reconstructed daily close then identifies a balance or ledger segment to examine. It does not establish that the bank charged an overdraft fee.

## 4.3 Update transactions and balances separately

**Update from CSV** imports additional records using the account's saved mapping. Overlap with a previous export is expected and is checked for duplicates. **Update balance** records a newer observed

balance. Importing transactions does not require replacing a correct anchor with an amount guessed from the export.

Loan and investment accounts accept dated balance histories, or supported activity exports integrated around an anchor. The activity route reconstructs principal changes; it cannot invent interest or valuation changes that are absent from the records. Use actual balance observations when available. Between observations the history uses interpolation, which should not be interpreted as an observed market price on every intervening day.

## 4.4 Investigate a difference

A reconciliation begins with a statement interval whose opening and closing balances are known. For posted amounts, the check is

$$B_{\text{close}} = B_{\text{open}} + \sum \text{deposits} - \sum \text{withdrawals}. \tag{4.1}$$

Include fees, interest, transfers, refunds, and business expenses that moved the account. Their exclusion from a spending score does not exclude them from the bank ledger.

1. Confirm the account, balance date, and posted closing figure. Compare dates as well as amounts.
2. Open Transactions for that account. Enable the running account-balance column when needed.
3. Remove display filters that hide relevant records, while remembering that the balance calculation itself uses the full ledger.
4. Compare the interval's opening balance, net posted flow, and closing balance with the statement.
5. Inspect the earliest date where the difference appears. Check missing rows, repeated imports, reversed signs, posting-date migrations, and duplicated transfers.
6. Use the balance-check and duplicate-review tools to examine candidate discrepancies. Review proposed removals before confirming them.

An unexplained difference should remain unexplained until its source is found. Adding a balancing transaction merely to make a chart agree conceals whether the original issue was an omitted fee, an incorrect anchor, or a duplicate posting.

## 4.5 Read running balances carefully

The balance shown after a transaction is reconstructed from the complete ledger. Search, category, time-period filters, and paging do not redefine that history. A filtered list can therefore jump between displayed balances because intermediate transactions are hidden.

Entries on the same date can have an unknown bank posting order. A balance marked **approximately** or with ≈ reflects an ordering qualification. An intermediate negative row does not establish a negative closing balance for that day. Use the daily-close check and the statement when investigating an actual overdraft.

Pending or unposted CSV entries are retained separately from the posted ledger. A missing pending item in Doug is not evidence that it is safe to spend the corresponding money. The application has no live connection through which to discover a hold omitted from an export.

## 4.6 Edit or remove an account

Review the selected account's name and type before editing it. An account deletion affects its associated records and can remove evidence used by historical calculations. Goal savings previously located in a deleted account become unlocated rather than being assigned to another account automatically. Make a backup before a large restructuring, and use Undo promptly if a deletion was unintended.

Implementation reference: `finfore/ui/accounts.py;finfore/ui/account_dialog.py;finfore/history.py;finfore/store.py;tests/test_posted_ledger.py`

Chapter 5

# Importing statements

## 5.1 Prepare the export

Download a CSV transaction export from the bank. A PDF statement is useful for reconciliation but is not the CSV input expected by the importer. Save the original export so that a later mapping or reconciliation question can be checked against the supplied records.

Import each account into its corresponding Doug account. Before importing an extended history, back up the database. When checking an unfamiliar bank format, begin with one file and inspect its result before applying the same choices to several accounts.

## 5.2 Map the columns

Choose **Import CSV** from the header or File menu, select the account, and choose the file. Doug proposes a mapping from column names and sample contents and summarizes the result under **What Doug read**: the number of transactions, the first and last dates, how many are money out and money in, any pending rows left out, and the first three rows as they will be saved. Compare that summary with the statement. The column choices are behind **Adjust how the file is read**. They open by themselves only when the file cannot be read, and an amber warning appears when the result looks implausible, for example when nearly every row reads as money in. The proposal requires review because banks use different signs, dates, delimiters, and headings.

| Field | Check before importing |
|---|---|
| Posting / balance date | Use Posting Date when supplied. Verify whether day-first parsing is appropriate. |
| Transaction date | Preserve the purchase date separately when supplied; it need not equal the posting date. |
| Description | Select the merchant, payee, or narrative rather than a numerical reference alone. |
| Amount | Choose signed amounts, reversed signs, separate debit/credit columns, or an indicator column as appropriate. |
| Status and reference | Retain supplied posting status and source reference when available. |

Inspect recognizable examples: a paycheck should increase the account; an ordinary purchase should decrease it. For a card export, do not infer the correct choice merely from its account type. The export may use positive purchases while Doug's canonical transaction amount is negative for an outflow.

## 5.3 Import and inspect the result

1. Review the account and the **What Doug read** summary. Open **Adjust how the file is read** to check the mapped dates, amount mode, and preview rows when anything differs from the statement.
2. Confirm the import only after recognizable amounts have the correct direction.
3. Read the import result, including added and skipped records or any explanation requiring attention.
4. Compare several imported rows and the date range against the original export.
5. Open Transactions and check the account's reconstructed balance against a known statement interval.
6. Review uncertain categories and detected recurring items before using the forecast.

An approved import can be taken back with Ctrl+Z or **Undo** in the header while the application remains open. An import also records a checkpoint for later comparison. This is a saved model state, rather than a substitute for inspecting the imported ledger.

## 5.4 Overlapping exports and identical purchases

An updated bank export commonly overlaps an earlier one. Doug uses import identities and transaction contents, including occurrence counts, to reduce repeated entries. Two genuinely identical purchases on the same day must remain two purchases; overlap detection therefore cannot be equivalent to dropping every repeated amount and description.

Differences in posting dates or meaningful merchant descriptions can still require manual review. Use the duplicate-review tools to inspect candidates, rather than assuming either that every repeated row is wrong or that every overlapping import is perfect. A remembered removal can prevent an entry previously identified as a duplicate from returning.

## 5.5 Correct earlier date mappings

An older import may have used Transaction Date when a newer export supplies both Transaction Date and Posting Date. The update path can show exact matching records whose ledger dates would change. Review this table before accepting **Update dates and import**. The purpose is to preserve the record's identity and associated review or goal information while changing its ledger date.

Ambiguous matches require review. A row already present on both candidate dates cannot be migrated safely by guessing which copy is correct. Wider date comparisons in duplicate review are candidates to examine, rather than instructions to remove everything shown.

## 5.6 Pending entries and malformed rows

Rows identified as pending or unposted remain outside the posted balance and spending calculations. The latest-import view shows the pending information supplied by each account's most recent export. A new export replaces that snapshot; it is not a live bank feed.

A malformed row or an invalid financial date or amount should be corrected rather than silently interpreted. Read the import explanation, check the original file, and obtain a clean export when

necessary. Do not remove a financially meaningful row merely to make the importer proceed. If a mapping was wrong, Undo the approved import or work from a backup, then re-import with the corrected choices.

## 5.7 Loan and investment imports

For these account types, the same import route opens balance-history handling. A file can provide dated balances directly, or supported activity from which a history is reconstructed around the entered anchor. The two shapes answer different questions: observed balances report values; activity reports changes that may omit interest, price movements, or other adjustments.

Inspect the balance history after import. An account whose latest observation is old may have a plausible interpolated chart without having a current valuation. Update the observation before treating that value as today's financial position.

Implementation reference: `finfore/ingest.py;finfore/pipeline.py;finfore/store.py;finfore/ui/import_dialog.py;finfore/dedup.py`

Chapter 6

# Reviewing transactions and categories

## 6.1 Review the evidence behind a category

Open **Accounts** > **Transactions**. Use the account, period, search, and review controls to find the records of interest. A category can be suggested by a keyword, a remembered merchant rule, or another automatic assignment. The existence of a category does not mean that the user has checked it.

The review state alongside a description distinguishes confirmed entries from entries requiring attention or ordinary unreviewed assignments. **Needs attention** identifies records worth examining; it is not a declaration that every displayed category is wrong. **Not reviewed** is broader and includes apparently ordinary assignments that have not been individually confirmed.

1. Select a transaction and compare its description, account, amount, and date with the export.
2. Open Transaction details to inspect the category source, review history, and balance information.
3. If the category is correct, use **Mark reviewed**. If it is incorrect, choose the appropriate category.
4. Review whether the change should apply only to these records or become a rule for future imports.

Changing a category or confirming it records a human review. Financial amounts and category changes are different operations; a correct category cannot compensate for an incorrect sign or a missing transaction.

Importing an overlapping CSV retains existing categories, review status, review dates, and one-off business decisions. The import summary distinguishes previously reviewed entries kept from new entries needing review. A remembered category can classify a new purchase, but that new transaction remains unreviewed until confirmed. Automatic category updates cannot replace a human-reviewed decision.

## 6.2 Keep one-off decisions separate from merchant rules

By default, a correction applies to the selected entries. Enable the preference for remembering category changes only when the merchant should normally use that category in future imports. A grocery order from a general retailer does not establish that all purchases from that retailer are groceries.

**Mark reviewed** confirms the selected entries even when the remembering preference is enabled. It does not automatically turn every confirmation into a merchant-wide rule. Conflicting category choices for the same merchant cannot be remembered as one unambiguous rule in a single batch.

Several rows can be reviewed or corrected together. Inspect the selection before applying an action, particularly when filters have changed. Category changes, their review evidence, and associated preferences are saved together and can be undone.

## 6.3 Review merchants in groups

The uncategorized-merchant review tool groups candidate records by merchant. Select a merchant to inspect the affected transactions and choose a category. Suggestions begin unselected; their presence is not an instruction to accept them.

The default scope concerns unknown records in the chosen period. An option can extend the review to older matching entries. Known or human-reviewed entries are preserved by that flow. Check the proposed number of merchants and transactions before saving, and distinguish a purchase from an incoming payment with a similar description.

For a name that already has a category, select one transaction and choose **Review same name**. The action appears beside Mark reviewed, in Tools, and in the row's context menu. Its preview lists exact matching names across all accounts and dates, ignoring case and extra spaces. Store numbers and other changes to the name remain distinct. Existing reviews are locked and preserved.

Only unreviewed entries in the selected transaction's category start checked. Choose the category for the checked entries, inspect their dates and accounts, and uncheck exceptions before applying the review. **Select this category** adds matching categories to the selection; **Select uncategorized** adds entries without a resolved category; **Clear selection** removes the checks. For example, Chipotle purchases can be reviewed together as Dining. Delta purchases classified as Travel and Business (reimbursable) can be reviewed in separate selections without changing each other's categories.

Remembering is off by default. Remembered rules can also match variants of a merchant's name beyond the exact-name preview. If those rule matches include conflicting categories, business entries, or a mixed reimbursement, this preview cannot create a future-import rule. Checking or correcting the selected transactions still records their individual reviews. Applying a batch is one undoable action; changing a transaction after the preview was opened requires the preview to be reloaded before that selection can be saved.

Similarity scores measure the match produced by the method; they are not calibrated probabilities that a category is correct. A close name match with conflicting previous labels is a reason for further review.

## 6.4 Recognize internal transfers

Moving money between two accounts belonging to the same household changes the location of holdings. It is not new income or personal spending. Review transfer and card-payment pairs carefully, especially when only one side has been imported.

Equal amounts alone do not establish an internal transfer. An external transfer can be a genuine payment. Conversely, categorizing both sides of a card payment as spending can count one economic purchase twice: once when the card purchase occurred and again when the card was paid.

An internal transfer remains part of each account's actual ledger even when excluded from household spending. The account balance calculation needs both the inflow and the outflow at their respective accounts.

## 6.5 Separate reimbursable business activity

Use **Business (reimbursable)** for expenses incurred for an organization and for a separate repayment of those expenses. These amounts are excluded from personal spending analytics and sampled behavior. They still change the actual bank balance until the repayment occurs.

When repayment is included in a paycheck, retain the deposit's income category and use **Business reimbursement** to identify the repaid portion. If a $3,500 deposit contains $500 of repayment, personal income is $3,000. The bank ledger retains the full $3,500; only the personal interpretation changes. Entering zero clears the reimbursement split.

A business label does not create a future repayment date. If an expected repayment is to be included in a scenario, use an explicit future income event and review it when the real repayment is imported.

## 6.6 Export reviewed data deliberately

The reviewed-data export is a local CSV for subsequent examination. It contains current human-reviewed labels and their associated dates, amounts, descriptions, account types, and review evidence. It excludes automatic guesses, unverified old labels, unresolved categories, future records, and the business or mixed-reimbursement cases excluded by that export's rules.

The export covers the relevant ledger rather than only the rows currently visible through display filters. Exporting does not train a model or send records to a service. The file still contains financial descriptions, so choose and protect its location accordingly.

Implementation reference: `finfore/ui/transactions.py;finfore/ui/merchant_review.py;finfore/categorize.py;finfore/learning.py;finfore/store.py`

Chapter 7

# Bills, paydays, and the calendar

## 7.1 Review a detected schedule

Open **Bills & payday** > **Detected bills & pay**. Doug groups repeated transactions and proposes amounts and cadences. Review the name, amount, frequency, most recent occurrence, and whether the pattern is still active. A recurring amount is normally based on the median observed payment, so an unusual recent bill may differ from it.

The available cadences distinguish every two weeks from twice a month. Every-two-weeks payments follow a fourteen-day interval. Twice-monthly payments follow two positions within a month and do not necessarily remain fourteen days apart. Monthly-family dates are clamped to short months when the requested day does not exist.

Deactivate a schedule that no longer applies. Use Ignore for a repeated item that should be withheld from the forward projection. Known pattern records, including disabled or ignored ones, are excluded from variable-spending sampling to avoid replaying the same historical item through a second route.

## 7.2 Link a loan payment

A loan-payment pattern must identify the loan to which it belongs. An unlinked pattern is excluded from direct scheduled replay to avoid counting a payment twice. When active linked patterns supply a loan's payment, their monthly equivalents replace the account's fallback payment; they are not added to the fallback again.

Check both sides of this relationship: the recurring item should have the appropriate sign and cadence, and the linked account should be the correct loan. Several linked payments can contribute to one loan's monthly amount. When no linked payment is available, the account's configured payment provides the fallback. Missing payment information can produce a model assumption or a warning, rather than evidence that no payment is due.

## 7.3 Read the calendar as a schedule

The Calendar tab lists future scheduled items through the next year, starting tomorrow. Select a day to see its entries; use **Show whole month** to restore the full monthly list. A date's color indicates income, payments, or both, while the entries supply the actual names and amounts. Today is marked by a ring inside its cell, whichever day is selected.

The schedule combines recurring items, configured loan commitments, investment contributions, and active saved what-if event occurrences. These sources are not all bank-confirmed payments. A saved hypothetical expense appears because it is part of the scenario records, not because a bank has scheduled it.

## 7.4 Check the payment account

Choose an account to inspect scheduled items assigned to it. A recurring item is attributed to an account only when the matching imported ledger supports a unique assignment. A merchant used on several accounts can remain unassigned. A manually configured contribution or payment may also have no confirmed funding account.

1. Review the combined schedule for the selected month.
2. Inspect **Unassigned items**. Determine which commitments require a payment-account decision outside the application.
3. Select a particular account and review the items attributed to it.
4. Compare their timing and amount with actual arrangements at the bank or payee.

For an account, the scheduled balance begins from its current recorded or reconstructed balance and applies assigned scheduled items through the selected month end. Later months include intervening scheduled items. A negative card position continues to represent money owed.

**A calendar balance is not a safe-to-spend amount**

The account-specific calendar excludes everyday variable spending, future transfers, and unassigned items. Loan commitments are listed as a schedule rather than stopped by the forecast's payoff calculation. A positive scheduled month-end amount therefore does not establish that the account can support all spending during that month.

## 7.5 When circumstances change

Review schedules after a pay change, move, cancellation, loan payoff, or a change in which account pays a bill. Importing additional data can improve detection, but it does not remove the need to confirm that the detected arrangement still exists.

The Forecast history option can re-estimate bills and paydays from a selected period without editing the saved pattern records. This forecast-specific refit does not rewrite the baseline calendar. Similarly, moving a payment date in a scenario changes the modeled cash flow; it does not contact a payee or change an actual bank instruction.

Implementation reference: `finfore/recurrence.py`;`finfore/calendar.py`;`finfore/ui/recurring.py`; `finfore/ui/calendar_page.py`

Chapter 8

# Reading Home and spending

## 8.1 Begin with the balances and their evidence

Home's Overview opens on its answer. A verdict sentence states where cash is most likely headed over the next 90 days, with its range, the next bill and payday, and a reminder that it is an estimate. With less than four weeks of history it says instead that it is too early to say, and offers to import more; when the newest statement is more than 30 days old it says it is going by older statements and names the last date it uses. The figures follow as an equation: net worth equals cash plus investments minus what is owed. Saved from income, beside them, is the share of income kept in a typical month. The net-worth history comes next, and **Needs a look**, which lists at most three matters across the program with a link to each, comes last. Read these components together: cash, investments, debts, and net worth describe related quantities. A high net worth can coexist with little readily available cash.

The history plot can show combined totals or separate selected accounts. Account and date selections affect the chart being inspected; they do not remove the other accounts from every calculation. Check the chart's active selection before comparing its value with a headline total that includes all accounts.

Layer controls and Expand change how information is displayed. If a reconciliation notice appears, open its details and trace the amount to the relevant account and dated observation. A cleaner-looking curve is not a reason to dismiss a balance difference.

## 8.2 Inspect monthly amounts before the grade

The Spending tab groups personal activity into Needs, Wants, Giving, and Uncategorized spending, with income treated separately. Subscriptions are shown distinctly within wants so that they can be examined without being counted twice.

Monthly amounts are recency-weighted averages of completed calendar months. The current unfinished month is excluded. Up to twenty-four recorded completed months contribute, with a three-month half-life: a month three months older has half the unnormalized weight. These figures answer a different question from a report for one named calendar month.

1. Read income and the spending groups as amounts per month.
2. Open **How this works** and its monthly breakdown.
3. Expand a group and category to inspect the contributing transactions.
4. Review the months and exclusions supporting the average.
5. Correct misclassified or missing records before interpreting an apparent spending pattern.

Refunds reduce their corresponding spending group. Internal transfers and reimbursable business activity are excluded from personal analytics, while a reimbursement embedded in income is removed

from the personal portion. These exclusions do not remove the underlying transactions from the bank ledger.

## 8.3 Interpret the score as a benchmark

The score combines a wants-to-needs comparison with a savings-rate comparison. Its implemented reference points are associated with the 50/30/20 budgeting benchmark. The exact interpolation tables and combination are given in Chapter 15; the result is not a personalized spending requirement.

A favorable wants-to-needs ratio can coexist with spending above income. For example, if needs and wants rise together, their ratio can remain unchanged while the household moves into deficit. Read the income shares and actual surplus alongside the score. Giving is shown as its own spending group and is not folded into the wants-to-needs numerator.

Large uncategorized spending makes the classification-based score provisional. An income or essential-spending denominator that cannot support a ratio can make a score unavailable. A missing score is different from a score of zero.

## 8.4 Trace a category to its ledger

Monthly cash flow and Spending by category are folded at the bottom of the Spending tab; open them to see the months and categories behind the averages. The cash-flow chart keeps its vertical scale fixed to the data, so dragging moves through time only. Open the spending breakdown or select a category bar to inspect the contributing records. The visible category chart and the weighted monthly breakdown may cover different stated periods; use the period named by the view. A recent ninety-day category list is not the complete evidence set for a twenty-four-month weighted average.

Transaction details can explain the original bank amount, the portion retained as personal spending, and its contribution to the monthly estimate. This distinction matters for refunds, reimbursements, and a recent large purchase. The bank amount answers what moved; the weighted contribution answers how it enters the estimate.

## 8.5 Turn an observation into an explicit plan

A spending suggestion does not itself alter a budget or a bank payment. Use Monthly budget to enter a changed amount, then decide how the resulting room should be allocated. Review the proposed change against commitments and recent circumstances rather than treating the score as a complete description of what is affordable.

If spending fell recently, distinguish a sustained change from a short quiet period. Forecast history can restrict the evidence to the relevant period, and Track record can compare earlier and later weekly spending. Neither tool proves that the same change will persist.

Implementation reference: `finfore/ui/dashboard.py;finfore/ui/insights_page.py;finfore/ui/spending_details.py;finfore/insights.py`

Chapter 9

# Goals and the location of savings

## 9.1 Choose what the goal measures

Open **Plan** > **Goals**. A goal can represent savings for a purchase or reserve, a cash-balance milestone, or a net-worth milestone. The choice determines the source of progress.

| Goal type | Meaning of progress |
|---|---|
| Savings for a purchase or reserve | Money explicitly set aside for this goal, plus qualifying purchases already recorded toward it. |
| Cash balance milestone | A priority-based share of household net cash; this is not a separate earmark. |
| Net worth milestone | A priority-based share of assets less debt, which can include investments and goal savings. |

Use a savings goal for a baseline that should have its own identifiable funds. A cash milestone can measure a household balance, but does not identify which account contains a protected reserve. Milestones may overlap savings already earmarked for another purpose, so they are shown separately in the funding map.

## 9.2 Create a savings goal

Choose **Add goal**, enter a recognizable name and goal amount, and select the progress type. Enter Already saved only for money that has actually been set aside. Creating the goal does not move that money or create an additional asset.

An opening saved amount without a source remains **Account not assigned**. After saving, select the goal and use **Locate savings** to assign all or part of that already-counted amount to an account. Doug does not guess the account. If only part can be located, assign that part and retain the remainder as unassigned.

An emergency-fund helper sizes a proposed savings goal from recorded needs and the selected number of months. Check its amount and priority against your circumstances. The selected month count is a modeling input, not a conclusion that the resulting amount is sufficient for a household.

## 9.3 Read the funding map

At the top of Goals, **Where your goal savings live** connects recorded savings with their account sources. Account bars share a dollar scale, so equal lengths represent equal dollar amounts across accounts. Each goal's progress bar instead uses that goal's own target; a full $1,000 goal and a full $1,500 goal can both have full-width progress bars.

Selecting a savings goal or an account segment highlights its locations and selects it in the priority

ladder. Selecting a savings row in the ladder updates the map. Clearing the selection disables actions that require a selected goal. A loan or household milestone remains a ladder target without acquiring a savings-account allocation.

Solid progress represents located savings; patterned progress identifies savings without an assigned account. Recorded purchases contribute to progress but no longer appear as money held in account segments. Read the exact amounts and source names as well as the shapes.

**A purpose is not another balance**

If four accounts each contain $1,000, total holdings are $4,000. A $1,000 baseline and a $1,500 funded goal describe purposes within those holdings. They are not added to the $4,000. The complete worked allocation in §14.2, p. 37 locates the remaining $1,500 in a partially funded goal.

Investment accounts are included at their tracked value. A CD marked as Investment remains an investment in Forecast. The map does not sell it, unlock it, or infer withdrawal availability. If an account's value falls below its recorded earmarks, the shortfall remains visible rather than silently reducing the recorded goal allocations.

## 9.4 Record what actually happened

Open **Goal activity** for the selected savings goal. Choose an activity, its account or unassigned location, amount, and date. An optional transaction link supplies evidence for a contribution or purchase; an optional note records context.

**Set aside money.** Add a new earmark of existing holdings. This increases saved progress but creates no income or transfer.

**Locate existing savings.** Assign part of an already-recorded unlocated amount to an account. Total saved progress is unchanged.

**Spend toward goal.** Reduce savings still held and increase purchases recorded toward the goal. Buying the planned computer therefore need not make the purchase goal appear unfunded again.

**Release savings.** Reduce savings and progress. Use this when drawing on a reserve that should be rebuilt, or making its funds available for a different purpose.

The line under **Activity** states what the chosen activity does. To remove an entry recorded by mistake, select it and choose **Delete activity**. Its effect on the goal's saved and purchased totals is reversed as if it had never been recorded, and the deletion can be undone. Doug refuses when later activity depends on the entry, for example a purchase made from the money a contribution set aside; delete the later entry first. Deleting differs from Release savings, which records that money left the goal and keeps that history.

After activity exists, use these activities rather than directly editing the saved total or changing the goal's kind. This preserves the history that explains its funding. A failed save retains entered fields for correction or retry. If a write succeeded but the view could not refresh, the dialog distinguishes that result from a failed write; reload the records rather than repeating the contribution.

## 9.5 Understand the funding checks

A new contribution must fit both household unassigned funding and the selected account's unassigned balance. Household funding includes tracked checking, savings, and investments, after credit-card balances and existing goal claims. It is not a spendable-cash measure. A purchase or release cannot remove more than the selected goal bucket holds.

Linked records must have the appropriate account, sign, date, and remaining personal amount. One deposit can be divided among several goals only within its usable amount. A link does not itself create another bank transaction. Removing an imported transaction retains the activity's reference evidence; deleting an account makes its remaining earmarks unlocated.

## 9.6 Set the monthly priority

The priority ladder places loans and goals in one order. Regular loan payments continue as existing obligations. The Monthly allowance supplies additional funding, divided by Focus, Weighted, or Custom. Focus funds the first unfinished target. Weighted gives more to higher-priority unfinished targets. Custom uses relative shares, not fixed dollar commitments.

For a $350 allowance, shares of 100 and 100 give the same equal split as 1 and 1: $175 for each before target-completion effects. The arrow beside an input shows the calculated extra dollars in the first plan month. All-zero custom shares fall back to the first unfinished target. Existing loan payments remain separate from these extra amounts.

Use Higher and Lower to change priority. Selection is retained by target identity when the ladder is reordered or refreshed. Completion dates and rollover estimates change with the plan; they do not update actual saved progress. Add contributions or purchases only when they occur.

## 9.7 Delete a goal deliberately

Delete goal names the selected goal and explains that its activity history is removed. Keep goal is the default and Escape choice. A confirmed deletion and its priority update are saved together; Undo restores the goal, its activity, and priority state. Account balances are separate records, so deleting an earmark is not a transfer of money.

Implementation reference: `finfore/ui/goals_page.py;finfore/ui/goal_funding_map.py;finfore/ui/goal_funding.py;finfore/goals.py;finfore/funding.py`

Chapter 10

# Building the monthly budget

## 10.1 Separate the available amount from the chosen allowance

The Monthly budget tab estimates what income and commitments leave available. Goals chooses how much of that room to allocate through its priority ladder. An available amount and a chosen allowance can differ: money may remain unassigned, or the chosen allowance may exceed the budget's room.

$$R = I - (E + D + W + G + U + Q + V) - A, \tag{10.1}$$

where $I$ is take-home income; $E$ is needs excluding regular debt; $D$ is regular debt payments; $W$ is wants; $G$ is giving; $U$ is uncategorized spending; $Q$ is the irregular-expense reserve; $V$ is investment contributions; and $A$ is the goals and extra-debt allowance. A positive $R$ remains unassigned. A negative $R$ is an over-budget amount rather than extra available money.

## 10.2 Build a plan from recorded evidence

Open **Plan > Monthly budget**. Start with the detected monthly amounts and their source explanation. Needs and regular debt payments are separated so that a loan payment already included in recorded spending is not added a second time. The detected debt line uses the larger of recorded debt spending and current regular loan commitments. Investment contributions begin from account settings.

1. Check take-home income, needs, debt payments, wants, giving, and uncategorized spending.
2. Enable Override only for a line whose planned amount differs from its detected value.
3. Add an irregular-expense reserve only for costs not already represented in the spending lines.
4. Inspect the amount available before the goal allowance, then compare it with the allowance selected on Goals.
5. Adopt the budget's available amount as the allowance only if that is the amount you intend to allocate.
6. Save the spending plan and read the resulting unassigned or over-budget amount.

Without sufficient complete history, enter explicit overrides for income and the spending lines, including genuine zero amounts. A missing history is not evidence that a spending category costs zero. The manual-plan requirements ensure that the user supplies the missing assumptions before saving.

## 10.3 Adjust categories without counting them twice

Expand **Adjust individual spending categories** to alter dining, subscriptions, groceries, or another supported category. These lines are components of Needs or Wants. They are not additional expenses added above the group total.

A group override takes precedence over its component category overrides. To plan through individual categories, turn off that group's total override. Review the resulting total after changing a category, especially where refunds affected the historical contribution. The detailed calculation replaces the selected signed contribution before flooring the group at zero; summing only positive category displays can therefore differ from the true group calculation.

If dining is reduced by $ 100 and subscriptions by $ 50, the intended change makes $ 150 more room only if those changes actually reduce the relevant group total by that amount and no other line changes. Inspect the final equation instead of adding a category reduction to an already overridden group total.

## 10.4 Use the shared allowance

Monthly allowance on Goals is shared with the budget. It is not a second unrelated number. A manual choice is retained; **Use Plan / history** restores automatic use of the saved budget or, without a saved budget, the historical estimate. The budget can offer its available amount as an explicit choice when it differs from the current allowance.

Budget-row edits remain a draft until Save spending plan. Reload saved plan discards those unsaved edits. Review the allowance separately because it can be saved from Goals without saving every budget-row draft. The calculation in Forecast uses the saved plan, not an unsaved draft that happens to be visible on another tab.

## 10.5 Connect the plan with Forecast

Enable **Use Goals funding plan** in Forecast to use the saved budget's income, spending, and investment assumptions together with the shared allowance and priority-based loan funding. Scenario multipliers then apply additional changes on top of that plan.

An earmark leaves its underlying holding unchanged. An irregular reserve remains in cash until an outflow is modeled. Fully funding a purchase goal does not cause the forecast to spend its target automatically. To test the purchase itself, add an appropriate future event or ensure that its expected spending is otherwise represented.

If an imported historical purchase was paid from an already-built reserve, inspect whether its monthly budget treatment would reserve the same amount again. A reasonable adjustment depends on how that purchase is represented in the spending history and future plan; the application cannot infer the intended treatment from the goal's name.

Implementation reference:
`finfore/ui/plan_page.py;finfore/spending_plan.py;finfore/goals.py;finfore/forecast.py`

Chapter 11

# Forecasts and scenarios

## 11.1 Establish the baseline before changing it

Open **Forecast** > **Projection**. Confirm the starting cash and net worth against Accounts and Home. Check the Data quality line and use Review data for the supporting records. Identify the history period, recurring patterns, and whether the saved Goals funding plan is enabled.

In Simple view the page opens on its answer and the chart. **Try a change** opens a panel beside the chart with the income control and the three largest spending categories, plus any category already set away from 100%. The result updates by itself after each change, and **Hide changes** closes the panel without undoing anything. Everything view shows the full set of controls permanently, including What-if events, Forecast history, and the analysis panel. When Home's outlook calls for caution, its **See what could change this** button opens the same panel.

With less than four weeks of history the headline says it is too early to say where cash is headed. When the newest statement is more than 30 days old, the headline names the date the forecast is based on.

The model begins from current reconstructed holdings. It replays scheduled flows, models debt payments and investment changes, and samples eligible historical weeks for variable spending and irregular income. The calculation methods are developed in Chapter 17. The visible result is conditional on these inputs, not a statement that every future cost is known.

For an account carrying card debt, also read the recorded opening net-worth qualification in Chapter 17. The inspected forecast omits that negative card position from opening net worth. Compare the account components before treating the displayed total as the household's complete position.

## 11.2 Read the curve and bands

The displayed percentile curves summarize simulated paths at each date. P50 is the median at that date. P10–P90 spans the central 80% of modeled values at that date, while P25–P75 spans the central 50%. The bands do not mean that 80% or 50% of complete future trajectories must remain inside them at every date.

Cash and net worth answer different questions. Investment growth can improve net worth while contributions reduce cash. Debt repayment reduces cash and debt together, with interest affecting the result. Review both quantities before interpreting an improvement in one as increased room to pay a bill.

In Everything view, choose **Cash + net worth** to place both measures on one graph with a shared date axis and dollar axis. Blue represents cash; green represents net worth. The Show control can also isolate either measure. Changing the displayed measures preserves the current axes; use **Reset axes** to fit the selected data.

## 11.3 Check data quality as evidence

The review identifies matters such as old or future-dated balance observations, old transactions, a short history, apparently empty months, uncertain categories, and scheduled outflows without an assigned account. Open the relevant records and compare them with statements.

A quiet month can be legitimate. An old observation can still be moved forward by newer posted transactions. A flagged category may be correct. The checks are prompts for verification; they neither modify the records nor certify that the projection is accurate. A green or unflagged state cannot account for an expense that was never entered.

## 11.4 Compare one explicit change

1. Read the baseline assumptions and active saved events.
2. Change a single income or spending assumption first.
3. Enable the unchanged-controls comparison when available. It is drawn as a dashed line beside the solid result.
4. Compare the timing, amount, and uncertainty of the cash and net-worth results.
5. Inspect what changed before making another adjustment.
6. Use Reset scenario to restore temporary controls when finished.

When Use Goals funding plan is enabled, the unchanged-controls curve omits the saved budget and Goals funding so their combined effect can be examined. Other active inputs, including saved hypothetical events, still require attention. The comparison is not necessarily a universal "do nothing" household baseline.

The engine uses common random draws for comparable baseline and scenario runs. This reduces differences caused merely by drawing another random sample. It does not remove uncertainty about future behavior or establish that the proposed plan is achievable.

## 11.5 Use the saved budget deliberately

With Goals funding enabled, saved budget amounts replace the corresponding monthly references. Existing recurring dates and weekly spending patterns are scaled to those amounts where evidence exists. A planned amount without a usable historical reference is spread evenly across days instead of being assigned an invented bill date.

The Goals ladder supplies loan payments, including existing obligations, allocated extras, and payment rollovers. Separate extra-loan and rollover controls are disabled in this mode to avoid conflicting decisions. Other scenario adjustments remain additional assumptions. Without this mode, Forecast uses its independent extra-loan and rollover controls.

Goal earmarks retain their cash or investment holdings. Their schedule describes intended allocations; it does not create a sale, transfer, or future purchase. The forecast can therefore improve because a budget lowers spending while an earmark by itself changes no cash path.

Under Goals, **Cash to show** selects **Bank cash** or **Free cash after goal funding**. Bank cash includes money set aside for goals. Free cash subtracts recorded cash earmarks and, with Goals funding enabled,

planned additions to savings goals and the monthly Plan's irregular-expense reserve. For example, putting $ 200 aside for a vacation reduces free cash by $ 200, while bank cash and net worth retain that money. Reducing spending increases bank cash under the same income and payment assumptions.

Future savings-goal allocations are reserved on each future first of the month; the irregular-expense reserve accumulates daily at its monthly amount divided by 30.44. Goal savings without a recorded account are conservatively treated as cash claims. Earmarks in investment accounts do not reduce cash, and cash or net-worth balance milestones do not create additional earmarks. Recorded purchases remain goal progress rather than cash still reserved. Free cash can be negative when the intended commitments exceed cash holdings; this alone does not mean the bank balance is negative.

Cash-shortfall warnings always use bank cash, regardless of this display choice. Historical bank cash and bank-cash buffer lines are hidden in the free-cash view because they describe a different quantity. The display choice is saved separately for each database and does not change the funding plan.

## 11.6 Add a hypothetical expense or income

Expand What-if events. Enter a recognizable name, the amount per occurrence, and its date. A negative amount represents a cost and a positive amount represents income. Choose Once or a repeat interval and, for a repeating event, an optional inclusive end date.

An event of $ 50 per week means $ 50 on every occurrence, not $ 50 divided across the entire repeat period. Monthly-family repeats remain anchored to the original day: January 31 can become the last day of February and then March 31. Only occurrences in the relevant future horizon affect the projection.

Events are saved records and are retained by Reset scenario. Use the event checkbox to compare with and without an event, or edit or delete it explicitly. Both comparison curves normally retain active saved events. Remove or disable a hypothetical when its effect is already represented by imported behavior or another planned amount, to avoid adding the same expense twice.

## 11.7 Select a relevant historical period

In Everything view, Forecast history can use Automatic, a rolling period, a fixed starting date, or all available history. Automatic normally caps eligible weekly samples at 78. An explicit period includes complete Monday–Sunday weeks within its boundaries and the recorded ledger span. Quiet eligible weeks remain zero-spending observations; older weeks are not silently imported to repair a short selection.

Under four usable complete weeks, historical variable-spending sampling is omitted and a warning is shown. Explicit saved-budget amounts may still supply planned daily flows. Extend the period or import more records before treating the sampled history as a representation of ordinary variable expenses.

The option to re-estimate paydays and bills uses the selected period without rewriting saved recurring records. It preserves disabled status and loan links, but a short period may miss an annual or quarterly payment. Loan terms and investment assumptions remain account inputs.

## 11.8 Investigate a modeled cash dip

Read the shortfall description and its dates, depth, duration, and recovery. The probability of crossing zero alone cannot distinguish a brief small dip from a prolonged deficit. The nearby-flow explanation names scheduled items contributing around the modeled low point; it is not a complete causal account of a household's finances.

A timing experiment changes the scenario. It does not change a payment instruction at the bank or payee. Before relying on a shifted date outside the model, verify that the actual payment arrangement can be changed.

Implementation reference: `finfore/ui/forecast_page.py;finfore/forecast.py;finfore/forecast_history.py;finfore/data_quality.py;finfore/events.py`

Chapter 12

# Track record and reports

## 12.1 Save the claim before the outcome is known

A checkpoint records balances, the spending score, and a forecast at a particular time. Imports create checkpoints automatically, and Take a checkpoint now records the current state explicitly. A later comparison can then distinguish an earlier saved projection from a newly fitted projection that already knows later spending.

Open **Forecast** > **Track record**. Select checkpoints according to the comparison of interest and read the dates, coverage, and source records. A checkpoint is a summary record, not a complete restorable financial database. Backups serve the latter purpose.

## 12.2 Compare an earlier forecast with later balances

The path comparison shows an earlier forecast fan alongside balances reconstructed from later records. The fraction of evaluated dates inside the earlier bands is descriptive evidence about that comparison. It is not the fraction of all possible future households covered, nor proof that the next projection is calibrated.

Unexpected outcomes can have several sources: changed behavior, missing records, a corrected balance, different payments, or a model assumption that did not fit. Check which records and dates overlap before assigning the difference to forecasting error alone. Very short follow-up periods can be insufficient for a useful comparison.

## 12.3 Distinguish saved and reconstructed checkpoints

Reconstruct history builds retrospective checkpoints using the data and pattern fit available at each selected historical date. These can make an existing ledger useful for comparisons on a first session, but they are not forecasts that were actually saved at that earlier time.

The historical boundary matters. Current saved goals and today's budget must not be treated as decisions known by an earlier household. A fixed history start date later than a historical refit date can leave no training data, in which case the comparison is unavailable rather than filled using later information.

## 12.4 Interpret a spending-change result

The program compares eligible weekly discretionary spending before and after a checkpoint. Recurring bills are excluded from this particular behavior comparison. The result concerns the weekly observations retained by the method, not every financial aspect of the household.

Read the estimated dollar change and its uncertainty together. A statistically distinguishable difference indicates evidence under the comparison's assumptions, rather than proof of its cause. An inconclusive result means that the available weeks did not resolve the change at the chosen threshold; it is not evidence that nothing changed.

The method uses a permutation comparison and a bootstrap interval. Weeks must be sufficiently comparable for exchanging their labels to be meaningful. Seasonality, long trends, missing intervals, or a changed mix of records can affect that interpretation. Category movements help locate changes but are not each presented as independent significance tests.

## 12.5 Export a month in review

Choose **File > Save a month in review** and select the calendar month. The resulting self-contained HTML report describes that month directly. It is different from the recency-weighted monthly averages used to describe current spending habits.

Open the report in a browser and use its print function when a paper copy or PDF is needed. The exported file contains financial information, even though its styling is self-contained and it makes no external requests. Verify the open database and demonstration banner before exporting.

## 12.6 Maintain useful comparisons

Import consistently, update observed balances, and keep notes about major changes in circumstances. Preserve a backup before broad corrections. Historical checkpoints remain records of what was saved; correcting a current account does not rewrite every earlier saved forecast.

Clear checkpoints removes comparison snapshots rather than the account ledger. Read the confirmation carefully. Removing a checkpoint can eliminate the saved claim needed to compare an earlier forecast with a later outcome, so keep those records when they remain useful.

Implementation reference:
`finfore/ui/progress_page.py;finfore/progress.py;finfore/report.py;finfore/pipeline.py`

Chapter 13

# Backups, updates, and recovery

## 13.1 Back up the database that is actually open

Choose **File > Back up data**. Confirm the current database path in the status bar and the presence or absence of a demonstration banner. The backup contains the open database, including accounts, transactions, goals, recorded activity, checkpoints, and database settings.

A backup made while a demonstration is open contains that demonstration. Separate view preferences and the in-memory Undo history are not part of the database backup. Keep backup names and locations clear enough that their household, date, and intended purpose can be identified later.

The backup uses SQLite's backup mechanism rather than an arbitrary copy of an actively changing database file. Retain backups independently of the only working copy. Whether a storage location is encrypted, synchronized, or accessible to others depends on the user's system and service configuration.

## 13.2 Restore without overwriting the only copy

1. Identify the intended database path and the backup to restore.
2. Close Doug so that the database is not in use.
3. Keep a separate copy of the current database, even if it appears incorrect.
4. Replace the intended working database with the backup, or open a separate copy using the source application's explicit database option.
5. Reopen Doug and verify the path, account names, most recent records, and several known amounts.

Restoring replaces the database state. It is not an import that merges the backup with newer transactions. An old backup may lack later imports or changes, so retain those original exports and reconcile what must be re-entered.

## 13.3 Use Undo within a session

Edit > Undo reverses the most recent recorded financial action; Redo reapplies an undone action when available. The header can also identify the pending Undo action. Keyboard shortcuts follow native conventions, including Ctrl+Z and supported redo alternatives.

An action can include several related database changes. For example, deleting a goal includes its activity and priority state. Undo operates on the saved action boundary rather than requiring a separate reversal for every affected row. The history is bounded in memory and ends when the application closes, so it cannot replace a persistent backup.

When a save fails, read the visible result before retrying. A failed activity write retains its entered fields. A saved activity followed by a failed view refresh is reported separately so that the same contribution

is not unintentionally recorded twice. Use Refresh to reload a view; use Record activity only for an activity that has not already been saved.

## 13.4 Update an installed application

Check for updates queries the project's GitHub releases. An update must have an appropriate published installer and checksum; a changed source checkout or a pushed commit alone does not update another installed copy. After an installation, inspect About Doug to confirm the version actually running.

Keep a backup before an update involving significant data changes. If an update check cannot reach the service, the current local database and ordinary offline application functions remain separate from that network operation. A failed check is not evidence that financial data was lost.

This manual has its own revision and inspected source baseline. An installer version and a manual revision are different identifiers. Verify both when reporting a discrepancy between a described control and an installed edition.

## 13.5 Recover from an unexpected result

1. Record the page, selected account or goal, date, and visible message.
2. Check whether the issue concerns a bank record, a plan, a temporary scenario, or display state.
3. Compare the relevant values with statements or the saved plan.
4. Use a provided Retry or Refresh when the problem is a failed load or save. Check access to the database file if it is locked or unwritable.
5. Preserve entered values and the original records before making a broad correction.

If reporting an application error, include the version, the steps that produced it, and the relevant message. Diagnostic details can include file paths or transaction descriptions. Review them before sharing, and prefer fictional reproduction data when the problem can be demonstrated without financial records.

The troubleshooting tables in Appendix B connect common symptoms with checks and recovery steps. They identify possibilities rather than claiming to diagnose a bank or user error without evidence.

Implementation reference:
`finfore/store.py;finfore/undo.py;finfore/update.py;finfore/ui/main_window.py;run.py`

# Part III

# Worked examples

Chapter 14

# A worked household example

The following examples use fictional accounts and records. Their purpose is to connect the task steps with independently checkable amounts. The accompanying script runs the calculations in temporary databases and retains their inputs, results, and inspected source hashes. It does not open the user's database. The examples are separate small cases; the forecast case is not a reconstruction of the four-account case.

## 14.1 Check a dated balance

Suppose an account's posted closing balance on January 3 is $ 1,000. Its ledger contains a $ 500 deposit on January 1, purchases of $ 100 and $ 50 on January 2 and 3, and a $ 25 purchase on January 4. The $ 50 purchase is already included in the January 3 closing observation.

| Date | Posted flow | Calculated close |
|---|---|---|
| 2026-01-01 | $+500.00 | $1,150.00 |
| 2026-01-02 | $-100.00 | $1,050.00 |
| 2026-01-03 | $-50.00 | $1,000.00 |
| 2026-01-04 | $-25.00 | $975.00 |

Moving one day forward gives $1{,}000 - 25 = 975$. Moving one day backward reverses the January 3 outflow, giving $1{,}000 + 50 = 1{,}050$. The January 1 close is therefore $1{,}050 + 100 = 1{,}150$. Adding every transaction to the anchor again would double count flows that occurred on or before its date.

To reproduce the interpretation in the interface, enter the January 3 observation, import those dated flows, and compare the reconstructed daily closes. Search filters may hide rows, but must not change those closes. The program-generated table above agrees with the independent arithmetic.

## 14.2 Locate four thousand dollars

Checking, Savings, Money market, and a CD each hold $ 1,000. The money market is entered as Savings and the CD as Investment. Create four savings goals, then record these locations:

1. Locate the $ 1,000 Baseline in Checking.
2. Locate the $ 1,500 Funded purchase across Savings ($ 1,000) and Money market ($ 500).
3. Locate $ 1,500 toward House across the remaining Money market ($ 500) and CD ($ 1,000).
4. Leave Unfunded purchase with no money set aside.

| Goal | Target | Held | Still needed |
|---|---|---|---|
| Baseline | $1,000.00 | $1,000.00 | $0.00 |
| Funded purchase | $1,500.00 | $1,500.00 | $0.00 |
| House | $10,000.00 | $1,500.00 | $8,500.00 |
| Unfunded purchase | $500.00 | $0.00 | $500.00 |

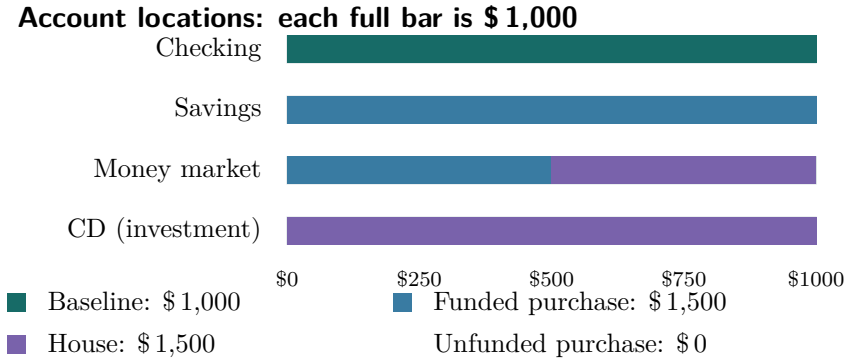



Figure 14.1: One set of holdings, divided among purposes. Account bars share a dollar scale. The $ 10,000 House goal is 15% funded even though its $ 1,500 occupies more account space than the fully funded Baseline.

The account total and located savings both equal $ 4,000.00. No additional money has been created. There is no unassigned holding in this example. The progress fractions are 100%, 100%, 15%, and 0%, respectively. A full goal-progress bar measures its own target, so the two full bars do not imply equal amounts.

If the initial $ 1,500 for House had been entered without locations, it would remain saved progress under Account not assigned. Use Locate savings to identify those sources; adding another contribution instead would count the same savings twice. Location does not make the CD withdrawable or change its Forecast account type.

## 14.3 Allocate a monthly allowance

Keep the first two goals complete and choose Custom for House and Unfunded purchase. With a $ 350 monthly allowance and equal shares of 100 and 100, the first month's allocation is

$$350\frac{100}{100+100} = 175$$

for each unfinished goal. Shares of 1 and 1 give the same result. These inputs are relative weights; entering 100 does not cap the allocation at $ 100. When the $ 500 target finishes, subsequent available money is redistributed among the remaining targets.

Now suppose $ 200 is spent from the CD toward House and the expenditure is recorded as Spend toward goal. House has $ 1,300.00 still held and $ 200 of recorded purchases. Its total progress remains $ 1,500. This activity changes the goal record only; an actual withdrawal must also appear in the bank or investment records. For a reserve withdrawal that should reduce progress and be rebuilt, use Release savings instead.

## 14.4 Make room in the budget

The following monthly budget makes the connection between a spending decision and a goal allowance explicit.

| Monthly line | Before | After |
|---|---|---|
| Take-home income | $4,000 | $4,000 |
| Needs, excluding debt | $2,500 | $2,500 |
| Regular debt | $0 | $0 |
| Wants | $700 | $500 |
| Giving | $100 | $100 |
| Uncategorized | $0 | $0 |
| Irregular reserve | $100 | $100 |
| Investing | $0 | $0 |
| Available before goals | $600 | $800 |
| Goal allowance | $350 | $550 |
| Unassigned | $250 | $250 |

Before the change, expenses total $ 3,400, leaving $ 600 before goals. An allowance of $ 350 leaves $ 250 unassigned. Reducing wants by $ 200 leaves $ 800 before goals. Choosing an allowance of $ 550 assigns the additional room while leaving the same $ 250 unassigned.

Saving a lower wants amount does not itself prove that spending will fall. The budget records the intended assumption. Confirm that the income and other commitments are complete, save the plan, and then test that assumption in Forecast.




Figure 14.2: The intended planning workflow. The allowance is shared between Goals and Monthly budget. Forecast uses the saved plan only when the corresponding option is enabled.

## 14.5 Compare the conditional forecast

The forecast example begins with $ 8,000 of checking on 2026-10-17. Its fictional history contains monthly pay of $ 4,000, rent of $ 2,000, utilities of $ 200, insurance of $ 100, giving of $ 100, and seeded weekly grocery and dining expenses. Pay and rent are saved monthly patterns. Their future dates remain in the forecast rather than being replaced by a straight monthly line.

The retained run uses 53 usable historical weeks, 600 simulated paths, a 365-day horizon, and seed 7. The historical grocery and dining generator uses seed 41. A second run enables the saved budget in the preceding table and its $ 550 allowance. There are no loans or investments in this small forecast case, so earmarking the allowance does not itself remove cash. The saved income and spending amounts are what change the cash path.

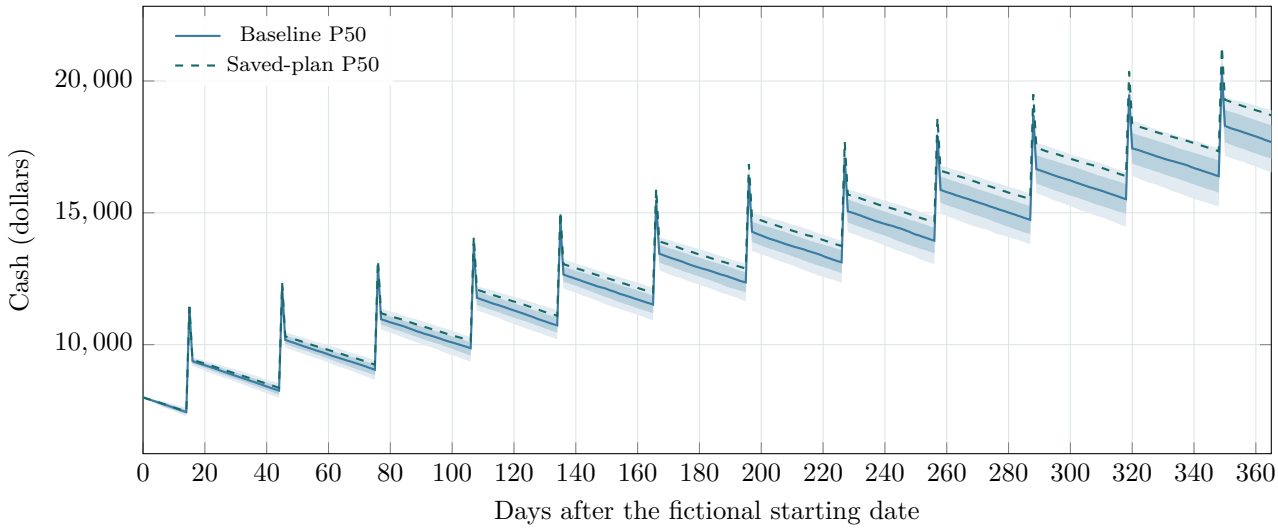



Figure 14.3: Fictional cash projection. The light and darker bands are baseline P10–P90 and P25–P75 at each date. Lines show the baseline and saved-plan P50. These conditional model outputs are not measured prediction accuracy.

The final baseline P50 is $ 17,684.20; the saved-plan P50 is $ 18,698.15. These values come from the retained run, not from a general claim that a particular budget always produces that difference. The planned case is compared with the synthetic historical baseline, whose category totals differ from the illustrative “Before” budget. It is therefore not a controlled experiment of the $ 200 reduction alone.

The generator checks the opening balance, percentile ordering, application of saved-plan mode, and exact repetition of the seeded planned P50. It also checks the account and goal arithmetic above. Those checks establish specific implementation properties. They do not establish that the synthetic history represents a household or that the forecast is accurate outside the model.

## 14.6 Repeat the examples

From `docs/manual`, run `make examples`, then `make pdf`. The generator records its date, dependency versions, source hashes, checks, and diagnostic comparisons under `generated/`. It uses the current date because the application’s ordinary entry points use the present day. A later date or source version can therefore change the retained forecast values even with the same seeds. Preserve the old manifest

if comparing editions.

The separate credit-card opening comparison in Chapter 17 is retained whether it agrees or identifies a limitation. A successful example build must not be read as a claim that every possible account or forecast behavior passed.

Implementation reference:
`docs/manual/examples/build_examples.py;docs/manual/generated/provenance.json`

# Part IV

# How the calculations work

Chapter 15

# Ledger and spending calculations

## 15.1 Reconstruct a cash ledger around an observation

Let $a_i$ be the signed amount of a posted transaction on day $d_i$, with deposits positive and withdrawals negative. Define cumulative posted flow through day $t$ as

$$F(t) = \sum_{d_i \le t} a_i.$$

If an observed closing balance $B_0$ belongs to day $t_0$, the reconstructed close is

$$B(t) = B_0 + F(t) - F(t_0). \tag{15.1}$$

At the anchor, the two cumulative-flow terms cancel and the result is exactly $B_0$. Before the anchor, later flows are reversed; after it, new flows are added. The balance is exact arithmetic on the supplied records when the anchor and ledger are correct. That qualification does not establish that an export contains every posting.

A later dated cash snapshot can supply the anchor instead of the original account observation. A future-dated observation is especially important to inspect because reconstructing today's balance around it requires future flows to be represented consistently. A date beyond today is not an additional source of observed current cash.

The application uses daily net flows for closes. Transaction-by-transaction running balances additionally require an ordering within each day. Where the source does not supply a reliable order, an intermediate balance has an ordering qualification even though the day's closing sum can be determined. Display filters do not change the ledger used by this calculation.

## 15.2 Combine account histories

Checking and savings balances contribute to cash. A credit-card balance is signed internally: an amount owed is negative and contributes debt; a positive card credit contributes cash. Loan balances contribute debt, and investments contribute assets. Historical net worth is assembled from those components rather than treating a goal's saved amount as another asset.

Loan and investment histories are interpolated between dated observations. Where the chart extends outside an account's known interval, an edge value may be carried flat. Similarly, cash history before its earliest known observation or transaction is filled at the edge rather than extrapolated through an unknown earlier ledger. A smooth or flat part of a chart can therefore be a reconstruction convention, not a sequence of measured balances.

## 15.3 Separate personal spending from ledger flow

Personal analysis excludes transfers and the business-paid portion of marked records. These exclusions do not remove the underlying ledger movement. A $3,500 deposit with $500 identified as business

reimbursement increases the account by $ 3,500 while contributing $ 3,000 to personal income.

Within known spending groups, refunds remain signed offsets. Unknown categories contribute negative personal flows to uncategorized spending. Income is drawn from positive personal amounts in the recognized income categories. These rules mean that reclassifying a transaction can change spending, the score, and the forecast sample without changing the account's posted balance.

## 15.4 Estimate a typical month

The monthly estimate considers up to 24 recorded complete months. The current partial month is excluded. A calendar month with no recorded personal rows is absent from this monthly sample; it is not automatically inserted as a complete zero month. This differs from the explicit weekly-window treatment used by Forecast.

Let $x_m$ be a signed monthly contribution to the group of interest, with outflows converted to positive spending and refunds retained as negative spending. If month $m$ is $k_m$ months before the newest included month, its weight is

$$w_m = 2^{-k_m/3}.$$

Thus a month three months older receives half the weight, and one six months older receives one quarter. The displayed nonnegative group estimate is

$$\bar{x} = \max\left(0, \frac{\sum_m w_m x_m}{\sum_m w_m}\right). \tag{15.2}$$

The zero floor is applied to the group, after signed contributions are combined. A category refund can offset another category in that group; flooring every category first would change the answer. The source-detail view traces individual weighted contributions and the included month shares.

## 15.5 Calculate the spending score

Let $I$ be monthly personal income, $S$ total personal spending, $E$ needs, and $W$ wants. The score uses the savings rate and wants-to-needs ratio,

$$r = \frac{I - S}{I}, \qquad f = \frac{W}{E}.$$

The implementation handles missing denominators rather than treating an undefined ratio as evidence of a good result. A wants-to-needs ratio alone cannot establish that spending is affordable: both needs and wants can be high relative to income.

The habit component $H(f)$ and means component $M(r)$ are linearly interpolated through the following anchors, with inputs outside each range clamped to its endpoints.

| $f$ | $H(f)$ | $r$ | $M(r)$ |
|---|---|---|---|
| 0.00 | 10.0 | −0.50 | 0.0 |
| 0.30 | 8.5 | −0.20 | 2.0 |
| 0.60 | 5.0 | 0.00 | 4.0 |
| 1.00 | 3.0 | 0.10 | 6.5 |
| 2.00 | 1.0 | 0.20 | 8.5 |
| 4.00 | 0.0 | 0.35 | 10.0 |

The combined value is

$$\text{score} = \text{clip}_{[0,10]}\left[0.6H + 0.4M - 2.5\left(1 - \frac{H}{10}\right)\left(1 - \frac{M}{10}\right)\right]. \tag{15.3}$$

For example, $f = 0.60$ and $r = 0.20$ give $H = 5$, $M = 8.5$, and a score of 6.2125 before display rounding. The interaction penalty is 0.1875. The formula is a chosen benchmark mapping; it is not a probability of financial success or a validated measure of personal behavior.

The score is provisional when more than 25% of spending remains uncategorized. Review the source records before using a category-sensitive result. Giving remains part of spending and the savings-rate calculation without being assigned to wants merely to complete a ratio. The application's descriptive wording and benchmark comparisons should be read with the actual amounts and the household's unmodeled commitments.

Implementation reference:
`finfore/history.py;finfore/categorize.py;finfore/insights.py;tests/test_posted_ledger.py`

Chapter 16

# Goal allocation and the budget model

## 16.1 Keep holdings, claims, and progress distinct

For savings goal $g$, let $s_g$ be money still held and $p_g$ purchases recorded toward it. With target $T_g$, displayed completion uses

$$c_g = \min\left(1, \frac{s_g + p_g}{T_g}\right)$$

for a positive target. Locating an unassigned saved amount changes its account attribution without changing $s_g$. A purchase moves an amount from $s_g$ to $p_g$; a release reduces $s_g$ without increasing $p_g$.

Account holdings supply the assets. Earmarks are claims within them. Funding capacity therefore subtracts existing goal claims when determining what remains unassigned. It also respects each selected account's remaining capacity. The household check includes investment values and card liabilities; it is broader than the cash immediately available for a bill. A market decline or later transaction can create an overfunding warning even if an earlier contribution fit the recorded balances.

Cash and net-worth milestones use separate household pots. Within each kind, higher-priority milestones consume progress before lower-priority milestones. Savings earmarks are not deducted from those milestone pots. Consequently, a cash milestone and a savings goal can refer to overlapping holdings. Adding their displayed progress would double count that overlap.

## 16.2 Build the monthly room

The budget calculation is Equation (10.1). Detected needs exclude the recorded debt component so that debt can be displayed separately. The detected debt amount is the larger of recorded debt spending and present regular loan commitments, rather than their sum. A user override replaces a detected line.

Individual category overrides replace their signed weighted contributions inside the relevant group. The group is then floored at zero. A group-total override takes precedence over component overrides. This precedence keeps an edited category from being added above an already chosen total.

The amount available before goals is $I - (E + D + W + G + U + Q + V)$. The chosen allowance $A$ is an independent decision. Automatic allowance uses the saved plan when available, or the historical personal surplus otherwise, floored at zero. A manually chosen allowance remains manual until automatic use is restored. The low-level allocation function also accepts an explicit pool; the interface supplies the resolved allowance. Saving a setting is not evidence that this amount is held today.

## 16.3 Divide the extra pool

Let $P$ be the current month's discretionary pool and $L$ the ordered unfinished targets. Regular loan obligations are outside $P$.

**Focus.** Offer the available pool to the first unfinished target.

**Weighted.** Give target rank $j$ weight $1/j$, then normalize over unfinished targets:

$$a_j = P\frac{1/j}{\sum_{k\in L}(1/k)}.$$

**Custom.** Normalize nonnegative relative shares $u_j$:

$$a_j = P\frac{u_j}{\sum_{k\in L} u_k}.$$

If all remaining shares are zero, use the first unfinished target.

An allocation is capped at the target's remaining need. Any amount left after a target finishes is redistributed among unfinished targets within that month. The first month's displayed extra allocation can therefore differ from its uncapped starting share. Multiplying all custom shares by the same positive constant leaves their normalized allocation unchanged.

## 16.4 Pay loans and release their commitments

For a loan with balance owed $D_m$, annual percentage rate $i$, and regular payment $q$, the monthly ladder first applies interest and the regular payment:

$$D_m^* = D_m(1 + i/12), \qquad q_m = \min(q, D_m^*).$$

Allocated extras then reduce the remaining debt. If the last regular payment requires less than $q$, its unused part is available in that month. After payoff, the former regular commitment enters later months' pool. Completing a savings target releases its share for other targets; it does not create an additional recurring income stream.

When a saved budget's debt line exceeds the ordinary loan minimums, the model can distribute the additional committed debt amount among loans before allocating the discretionary pool. The resulting required schedule remains part of the budgeted debt commitment. It should not also be entered as an independent extra-loan assumption.

A regular payment below monthly interest can leave debt growing unless extras make up the difference. A missing payment or a conflicting linked recurring payment is therefore a model input to resolve, not a reason to assume rapid repayment. The ladder uses a monthly APR model; lender-specific daily accrual, fees, rate changes, and exact statement conventions require separate evidence.

## 16.5 Interpret completion dates

The deterministic ladder starts in the current month and searches up to 360 months by default. Its dates assume that the allowance, regular commitments, priority, and terms continue as modeled. A target already complete is distinguished from an unfinished target outside the horizon. "Not reached within 30 years" is a bounded model result, not proof that a target can never be reached.

The schedule does not post future contributions to the saved goal history. Nor does it automatically buy the item when its goal completes. Forecast uses the loan schedule and, when enabled, the saved budget assumptions; actual savings contributions, account transfers, and purchases still need their

corresponding records. This distinction keeps an intended allocation from being mistaken for an observed transaction.

Implementation reference:
`finfore/goals.py;finfore/funding.py;finfore/spending_plan.py;finfore/store.py`

Chapter 17

# Forecast construction

## 17.1 Construct a daily path

The forecast creates a daily grid beginning at the selected origin. An ordinary run begins today; a historical comparison uses its specified earlier origin. Current reconstructed cash supplies the opening cash value. Future recurring flows, loan payments, investment contributions, and hypothetical events are applied to the appropriate future dates. Variable weekly net flows supply the remaining modeled behavior.

In schematic form, for path $k$,

$$C_{k,t} = C_0 + \sum_{u=1}^{t} \left( f_u + b_{k,u} \right),$$

where $f_u$ contains the scheduled cash effects and $b_{k,u}$ the sampled variable net flow. Investments and modeled loan balances then contribute to net worth. The calculation is conditional on which liabilities and flows the implementation actually represents.

**Recorded credit-card qualification**

The retained probe uses checking of $ 1,000 and card debt of $ 250. Ledger net worth is $ 750.00, while the inspected forecast begins at $ 1,000.00. The difference is a recorded implementation limitation, not an additional asset. Compare the forecast opening values with Accounts when carrying card debt; do not interpret this offset as available wealth. The probe is retained for maintainers to review after model changes.

## 17.2 Replay recurring items without sampling them twice

Active recurring pay and bills are scheduled using their saved cadences, amounts, and scenario adjustments. Monthly-family dates are clamped when the requested day does not exist. Linked loan payments are represented through the loan engine rather than also replayed as independent bills.

Known recurring keys are withheld from the variable-spending sample, including disabled patterns. Otherwise disabling a bill could simply cause the same historical bill to reappear as sampled variable spending. Review the detected patterns and their active status before treating the separation as complete.

An ordinary standalone loan model uses monthly APR interest, a payment day, and a payment capped at the remaining amount owed. A missing payment can lead to an assumed 60-month amortizing payment with a warning. This is a fallback model assumption, not a recovered lender term. With Goals funding enabled, the priority ladder supplies the loan-payment schedule, including committed debt amounts and extras. Separate extra-loan controls should not be used to duplicate it.

## 17.3 Sample complete behavioral weeks

Historical records are grouped into complete Monday–Sunday weeks after personal exclusions and removal of known recurring keys. Each retained row contains category spending and the corresponding irregular income. The engine samples whole rows so that categories and income from a given historical week remain together. It does not independently draw every category from a different week.

Automatic history retains at most 78 eligible weeks. An explicit period respects its selected boundary and the recorded span, including quiet eligible weeks as zeros. At least four usable complete weeks are needed to model this variable component. A shorter sample produces a warning and no sampled variable flow; it does not establish that future variable spending will be zero.

The weekly net amount is spread evenly over seven future days. Scheduled bills retain specific dates, while sampled weekly behavior does not preserve every original purchase's weekday. This resolution matters when interpreting a brief low balance.

## 17.4 Weight a sustained change and retain uncertainty in the mean

Weekly sampling is uniform unless a sustained-change gate is met. The inspected gate compares the recent third of weekly net flows with the earlier portion, requires at least 16 weeks on each side, a standardized mean difference of at least 0.8, and a permutation-test value below 0.01. When it engages, sampling weights decay with a 13-week half-life. This is separate from the always-weighted monthly estimate in Chapter 15.

The gate is an implementation choice for deciding when to favor recent observations. Detecting a past difference does not establish that it will persist. A changed export format, missing expenses, or seasonality can also change the observations.

Independent resampling alone would treat the estimated weekly mean as increasingly certain over a long horizon. The engine therefore draws one additional normal offset for each path and retains it for that path's entire horizon. Its scale is based on the standard error of the weighted weekly mean. For normalized weights $w_j$,

$$n_{\text{eff}} = \frac{1}{\sum_j w_j^2}, \qquad \text{SE} = \frac{s_w}{\sqrt{n_{\text{eff}}}},$$

where $s_w^2$ is the weighted net-flow variance with the implemented effective-sample correction. This term broadens long-horizon uncertainty within the model; it cannot supply missing categories or unforeseen structural changes.

## 17.5 Apply a saved monthly plan

With Use Goals funding plan enabled, the saved budget establishes monthly income, category spending, and investment targets. Recurring and weekly reference amounts are scaled to those targets, preserving dates and sampled relative variation where a usable reference exists. When no reference exists, a monthly amount is represented by a daily flow using 30.44 days per month.

Scenario multipliers apply on top of the saved plan. Debt is modeled once through tracked loans, or as a budgeted cash expense when no loan account exists. Investment contributions require an investment account; otherwise the unapplied amount is explained by a warning. The budget's reserve stays in cash rather than becoming a second automatic expense, and savings earmarks do not subtract their amounts from holdings.

The plan is disabled at historical forecast origins so that today's saved budget and Goals decisions are not treated as knowledge available in the past. Current saved hypothetical events are likewise excluded from an ordinary historical reconstruction. Such a reconstruction still depends on the records and historical assumptions retained by the application.

## 17.6 Model investments at monthly boundaries

An investment begins at its tracked value $V$. At each modeled month boundary, elapsed time $\Delta t$ is measured in years using 365.25 days. With annual return input $\mu$, annual volatility input $\sigma$, contribution $q$, and standardized innovation $Z$, the implemented stochastic update is

$$V_{\text{next}} = V \exp\left[(\mu - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}\, Z\right] + q. \tag{17.1}$$

The default innovation is a Student-t draw with five degrees of freedom, divided by $\sqrt{5/3}$ to give the innovation unit variance. The alternate configuration with degrees of freedom at or below two uses normal draws. The daily chart holds the monthly updated value between event dates, so this is a stepped model rather than a simulated daily market price.

The deterministic reference instead uses

$$V_{\text{next,ref}} = V \exp(\mu\Delta t) + q.$$

The same contribution reduces cash on that date. It is a transfer into the modeled investment, not a simultaneous loss of net worth of the full contributed amount. Budget-mode contributions are distributed according to existing account contribution shares, or to the first investment if none are set; extra investing also goes to the first investment.

The deterministic reference is not identified here as the exact stochastic expectation or median. In particular, the lognormal mean adjustment associated with normal innovations cannot simply establish the mean of an exponentiated Student-t model. Return and volatility inputs are user assumptions. The engine does not infer CD maturity restrictions, tax treatment, withdrawal penalties, or future allowed contributions from an account's name.

## 17.7 Summarize the paths

At each date, the engine calculates the 10th, 25th, 50th, 75th, and 90th percentiles across modeled paths. A percentile curve joins successive pointwise summaries; it is not necessarily one path that was simulated. The fan is conditional on this model and sample, and its nominal percentages are not measured forecast accuracy.

Shortfall summaries use complete paths to estimate crossing, low-point depth, time below zero, and recovery. They answer a different question from whether one date's P10 is below zero. A common random seed makes comparable scenarios easier to inspect by reducing random-draw differences, but adding accounts or changing model branches can alter the draw sequence. Seeded repetition establishes reproducibility for the same inputs and implementation, not correctness for all inputs.

Implementation reference: `finfore/forecast.py`;`finfore/forecast_history.py`;`finfore/recurrence.py`; `finfore/events.py`;`finfore/goals.py`

Chapter 18

# Evidence and interpretation

## 18.1 Distinguish four kinds of evidence

A bank statement supplies observations for reconciliation. A deterministic arithmetic check establishes whether specified records satisfy an equation. A software check establishes particular behavior for its tested inputs. A later forecast comparison provides evidence about agreement with subsequently reconstructed history. These forms of evidence answer related but different questions.

For example, a correct opening balance does not establish complete future spending. A reproducible Monte Carlo run can reproducibly apply an unsuitable assumption. A clear interface can make an incomplete ledger easier to inspect without making it complete. The manual keeps those boundaries explicit so that a result can be checked at the appropriate level.

## 18.2 Retain forecasts before evaluating them

A saved checkpoint preserves the earlier claim. Later records provide the comparison history. Evaluation requires a meaningful overlapping interval, correctly dated observations, and an understanding of which accounts and flows were included. A corrected anchor can alter reconstructed history even when the earlier checkpoint remains unchanged.

Retrospective checkpoints fit at an earlier boundary and can help inspect an existing ledger. They must remain identified as reconstructed. Excluding later rows and refitting patterns at that boundary reduces look-ahead, but it does not turn a retrospective run into a forecast actually saved at the time.

## 18.3 Interpret coverage descriptively

Track record can report the fraction of evaluated dates whose reconstructed values lie within an earlier pointwise band. Adjacent daily balances are dependent, and each comparison comes from a particular household interval. Coverage over those dates is therefore descriptive evidence, not a collection of independent trials establishing general calibration.

Read coverage together with the median error, direction of misses, follow-up length, and source records. A narrow band that repeatedly misses, a wide band that covers nearly everything, and an opening offset caused by a missing liability require different investigations. The opening credit-card probe in Chapter 17 is an example of an implementation qualification that should not be interpreted as an investment outcome.

## 18.4 Compare weekly spending before and after

For eligible discretionary weekly spending, the reported change is

$$\widehat{\Delta} = \overline{x}_{\text{after}} - \overline{x}_{\text{before}}.$$

A negative value means lower spending in the after sample. At least four complete weeks are required on each side. The inspected implementation uses 20,000 random permutations of the pooled weekly labels, compares the absolute mean difference with the observed absolute difference, and uses a 0.05 significance threshold.

The interpretation requires exchangeability under the null comparison. Strong temporal dependence, seasonality, changed recording practices, or unequal distribution shapes can complicate that premise. A label-permutation comparison of means should not be described as automatically valid for every combination of unequal variances and sample sizes.

A separate 4,000-replicate bootstrap resamples within each period and takes the 2.5th and 97.5th percentiles of the mean difference as a nominal 95% interval. This is an interval for the sampled comparison under its assumptions, not a bound on every future week's spending. The default random seed is 11. The dollar effect and interval usually matter more to a budgeting decision than the threshold alone.

An inconclusive threshold result does not establish zero change. A statistically distinguishable change does not establish that a goal, a button, or the application caused it. Category changes are descriptive contributions rather than a family of independently confirmed causal effects. Review the period definitions and records before interpreting the wording of a result more strongly than its method supports.

## 18.5 Understand what the manual checks

The retained example checks verify dated-anchor arithmetic, conservation of the four-account allocation, custom-share normalization, purchase-progress behavior, absence of an automatic ledger posting from goal activity, budget arithmetic, the score's worked anchor calculation, forecast opening cash, ordered percentiles, saved-plan application, and seeded repetition. The diagnostic credit-card comparison is retained separately from those pass assertions.

These examples do not replace the application's test suite or user review of statements. The document checker verifies references, included source files, generated artifact hashes, and PDF build results. Its optional source-drift check identifies application files changed since the examples were built. None of those document checks opens a real database or constitutes validation of future financial outcomes.

Implementation reference: `finfore/progress.py;finfore/history.py;docs/manual/examples/build_examples.py;docs/manual/examples/check_manual.py`

Appendix A

# Quick reference

## A.1 Minimum records for a useful first forecast

| Record | Supply and verify |
|---|---|
| Cash account | Type, observed posted closing balance, observation date, and a complete relevant transaction interval. |
| Credit card | Current debt or credit, posting dates, purchases, and payment transfers. Check the opening net-worth qualification in Chapter 17. |
| Loan | Current amount owed, APR, regular payment, payment day, and any recurring-payment link. |
| Investment | Current observed value, date, annual return and volatility assumptions, and monthly contribution. |
| Recurring item | Amount per occurrence, cadence, dates, active status, account assignment, and loan link if applicable. |
| Saved budget | Explicit intended amounts, shared goal allowance, and evidence that the saved version is the one being tested. |

## A.2 Canonical signed transaction example

The importer maps bank-specific headings to ledger meanings. The following is a small example of a signed export, not a universal required bank schema. Positive amounts are deposits and negative amounts are withdrawals.

```
Posting Date,Transaction Date,Description,Amount
2026-01-02,2026-01-02,PAYROLL,3000.00
2026-01-05,2026-01-03,GROCERIES,-72.48
2026-01-06,2026-01-04,STORE REFUND,18.00
```

The January 3 purchase belongs to the January 5 posted ledger in this example. Map both date fields when available. Check the preview's date interpretation, especially for a date such as 03/04/2026. Do not infer day-first or month-first from an ambiguous row alone.

Supported amount representations include a signed amount; a signed amount with its direction reversed; separate debit and credit columns; and an amount paired with an income/outflow indicator. Choose the mode that makes recognizable deposits positive and purchases negative in the preview. Currency symbols, grouping separators, parentheses, and other bank formatting still require successful parsing and review.

Loan and investment imports instead use dated balances or supported activity around an anchor. A transaction export without valuations cannot establish daily investment market value. The selected account type determines the import route.

## A.3 Signs in forms and stored records

| Item | Convention |
|---|---|
| Transaction amount | Positive entering the account; negative leaving it. |
| Card amount owed in the form | Normally entered as a positive debt; represented internally as a negative signed card balance. |
| Loan amount owed | Positive magnitude of debt. |
| Budget expense | Positive planned cost; the budget equation subtracts it. |
| What-if event | Positive income or negative cost per occurrence. |
| Custom goal share | Nonnegative relative weight; not dollars and not cents. |
| Annual return, volatility, APR | Percentage inputs in the interface; review the displayed percentage before saving. |

## A.4 Actions that are easy to confuse

| Action | Result |
|---|---|
| Reset axes | Restore the chart's default visible extent. |
| Reset scenario | Restore temporary scenario controls; saved events are managed separately. |
| Reload saved plan | Replace unsaved budget-row edits with the saved plan. |
| Use Plan / history | Restore automatic selection of the Goals allowance. |
| Locate savings | Assign already-recorded savings to an account without adding progress. |
| Set aside money | Add a new earmark within existing holdings. |
| Spend toward goal | Reduce savings held and retain the amount as completed purchase progress. |
| Release savings | Reduce both savings held and progress. |
| Update balance | Record an observed amount and date; it is not a missing bank posting. |
| Back up data | Save a restorable database copy; a checkpoint is a different record. |

## A.5 Keyboard and location reference

F1 opens Getting started. Ctrl+Z invokes Undo; supported redo alternatives include Ctrl+Y and Ctrl+Shift+Z. Native menu shortcuts may differ by platform. Tab and Shift+Tab move among controls, while arrow keys navigate suitable lists. Escape leaves a dialog or keeps an item when its confirmation uses that default. Check the visible selected row before applying a row action.

The default local database is `~/.ledgerline/ledgerline.db`. The current path is shown in the status bar; a source launch can select another path with `-db`. Separate view preferences are managed by the interface and are not a second financial ledger. A demonstration database remains distinct from the user's working database.

Implementation reference:
`finfore/ingest.py;finfore/models.py;finfore/ui/main_window.py;finfore/ui/widgets.py;run.py`

Appendix B

# Troubleshooting

Start with the smallest known discrepancy. Preserve the original export and observed statement amounts so that a correction can be checked. The possible causes below are checks to perform, not a conclusion about who made an error.

## B.1 Accounts and imports

| Symptom | Check and recovery |
|---|---|
| Balance differs from the bank | Compare posted closing amounts on the same date. Reconcile one known interval; inspect pending holds, missing fees, reversed signs, and duplicates. See §4.4, p. 11. |
| Transactions and Home differ | Confirm the account, selected date, full ledger, and anchor. Inspect future-dated observations and same-day order qualifications. A filtered transaction display does not redefine the balance. |
| Same-day row appears negative | Check the day's close and bank posting order. An approximate intermediate balance does not establish an actual negative daily close. |
| Paychecks decrease the account | Review the import amount mode. Undo the mistaken import or restore a safe copy, then import with the corrected sign convention. |
| Dates are reversed or displaced | Review ambiguous date parsing and Posting Date versus Transaction Date. Use the explicit matching-date migration when available; inspect ambiguous matches. |
| Updated export adds repeats | Compare original records and duplicate candidates, including occurrence counts and date differences. Preserve genuine identical purchases. Do not remove all repeated amounts automatically. |
| Pending purchase is absent | Check whether the latest export contains it and whether it is marked pending. Doug cannot discover an unexported hold through a live bank connection. |
| Investment history looks flat | Check dated observations, the account's known interval, and whether the export contains values or only activity. Flat fill and interpolation are not new valuations. |
| Import reports an invalid row | Read its reason and original CSV value. Correct the mapping or obtain a clean export; do not discard a meaningful financial row merely to pass parsing. |

## B.2 Goals and budgets

| Symptom | Check and recovery |
|---|---|
| Saved goal has no account | Select it and use Locate savings. This assigns already-recorded money rather than adding another contribution. |
| Goal funds exceed holdings | Review actual account values, earlier earmarks, and unlocated savings. A later withdrawal or value decline can invalidate earlier capacity. |
| Funded goal does not mean cash is available | Inspect account locations and account type. Investment earmarks and a CD's tracked value do not establish immediate withdrawal availability. |
| Purchase makes progress unexpected | Use Spend toward goal to retain purchase progress; use Release savings when the reserve should be rebuilt. Verify the separate actual account record. |
| Two custom inputs of 100 give more than $100 each | They are relative shares. Inspect the allowance, unfinished targets, and completion caps. Equal shares split $350 into $175 each. |
| Category cut does not change the budget | Check whether its Needs or Wants total is overridden. That total takes precedence over component categories. Save the intended plan after reviewing the final total. |
| Available amount differs from allowance | The former is calculated room; the latter is the chosen allocation. Adopt the available amount explicitly or retain an intentional unassigned remainder. |
| Allowance changes but draft rows do not | Goals can save the shared allowance independently. Save the budget-row draft separately; Forecast reads the saved plan. |
| Goal completion is outside the horizon | Review priority, allowance, remaining amount, loan interest, and minimums. A 360-month search limit is not proof of impossibility. |

## B.3 Forecasts and application state

| Symptom | Check and recovery |
|---|---|
| Saved budget has no forecast effect | Enable Use Goals funding plan. Confirm the budget was saved, inspect warnings, and check that scenario multipliers do not offset the intended change. |
| Goal earmark does not lower cash | An earmark changes purpose, not holdings. Model an actual future purchase with an event or the appropriate spending assumption. |
| Opening net worth differs with card debt | Read the retained implementation qualification in Chapter 17. Compare components with Accounts before interpreting the forecast. |
| Forecast omits ordinary variable costs | Check for at least four usable complete weeks, the selected historical period, recurring exclusions, and personal/business classification. |
| Forecast differs after changing history | Inspect the selected weeks, recurring refit option, detected change weighting, and retained warnings. A new training sample can change both center and width. |
| Reset scenario leaves an expense | Check saved What-if events. They are persistent records and must be disabled, edited, or deleted explicitly. |

| Symptom | Check and recovery |
|---|---|
| Historical comparison is unavailable | Check its origin, eligible complete weeks, selected start date, and follow-up overlap. Later data must not be used merely to fill an earlier training gap. |
| Spending change is inconclusive | Inspect the dollar interval and number of weeks. More consistent follow-up can help; an inconclusive threshold does not establish no change. |
| Control or tab is absent | Check Simple versus Everything, selected section, and installed version. Older releases use different navigation. On Forecast in Simple view, **Try a change** opens the scenario controls. |
| Screen is too bright or too dark | Choose **View** > **Appearance**. Match my computer follows the operating system's light or dark setting. |
| Chart keeps an old zoom | Use Reset axes to restore the current default extent. Ordinary graph changes preserve the inspection view. |
| Save or refresh fails | Read whether the write failed or the write succeeded but display refresh failed. Retain entered values; avoid repeating an already saved activity. Check file access and retry the appropriate operation. |
| Wrong data appears after reopening | Read the database path and demonstration banner. Open the intended working copy; do not merge or overwrite files before identifying them. |
| Update check finds no new installer | Check About and the published release when connected. Source changes and draft releases do not by themselves update an installed application. |

If the cause remains unknown, preserve a database backup and write down the version, page, dates, actions, and exact message. A fictional reproduction is preferable when it demonstrates the same issue. Review exported reports, logs, and descriptions before sharing them because they can contain financial information.

Appendix C

# Glossary and symbols

**Allowance** The chosen monthly amount for goals and extra debt funding through the priority ladder.

**Anchor** An observed balance and its date used to reconstruct a ledger backward and forward.

**Available before goals** Budget income less the specified spending, regular debt, reserve, and investing lines, before subtracting the allowance.

**Bootstrap** Repeated sampling from recorded observations to construct a modeled distribution or estimate an interval.

**Cadence** An item's repeat interval, such as weekly, every two weeks, semimonthly, or monthly.

**Checkpoint** A saved summary and forecast used for later comparison. It is not a restorable database backup.

**Complete month or week** An elapsed calendar period eligible under the particular calculation. Monthly averages and weekly forecast samples use different missing-period rules.

**Deterministic reference** A calculation without stochastic draws. It is not automatically the stochastic mean, median, or most likely complete path.

**Earmark** An explicit assignment of existing holdings to a savings purpose. It does not create another asset or a bank transfer.

**Effective sample size** The information-equivalent count used by the implemented weighting formula, $1/\sum w_j^2$ for normalized weights.

**Goal progress** Savings still held plus recorded qualifying purchases for a savings goal; a household-balance allocation for a cash or net-worth milestone.

**Located savings** Recorded goal savings assigned to a specific tracked account.

**Net worth** Assets less debt as represented by the relevant history or model. Compare the components where an implementation qualification applies.

**Override** A deliberate replacement of a detected budget amount. A group-total override takes precedence over category overrides inside that group.

**Pending** An unposted entry supplied by an export. It is kept outside the posted ledger and is not a live reservation of all possible bank holds.

**Percentile** A pointwise rank in modeled outcomes. P50 is the median at a date; a sequence of percentile values need not be one simulated trajectory.

**Permutation comparison** A comparison constructed by exchanging observation labels under an exchangeability premise.

**Personal amount** The part of a transaction retained after applicable business or reimbursement exclusions for personal analysis.

**Posted close** The account balance after all postings for the selected date, as distinguished from an available balance affected by holds.

**Provenance** The retained origin of an item or result, including source records, classification evidence, or generation inputs and hashes.

**Release** A goal activity that removes saved money and progress so that its use no longer counts toward the goal.

**Rollover** Reallocation after a target finishes, including regular loan payments freed after payoff.

**Scenario** A conditional set of changes to a baseline model, such as a spending multiplier or hypothetical dated expense.

**Unassigned holding** Tracked funding capacity not yet claimed by recorded savings goals. This is distinct from recorded savings whose account has not been assigned.

**Unlocated savings** Already-counted goal savings without an assigned account. Locating it changes attribution rather than progress.

**Weighted estimate** An average that gives observations different specified influence rather than treating them all equally.

## C.1 Symbols used in the calculation chapters

| Symbol | Meaning | Units |
|---|---|---|
| $B(t), B_0$ | Reconstructed close and observed anchor | dollars |
| $F(t)$ | Cumulative signed posted flow | dollars |
| $I, E, D, W, G, U, Q, V$ | Budget income, needs, debt payments, wants, giving, unknown, reserve, investing | dollars/month |
| $A, R$ | Monthly allowance and remaining budget amount | dollars/month |
| $T_g, s_g, p_g$ | Goal target, savings held, and recorded purchases | dollars |
| $P, a_j$ | Monthly discretionary pool and target allocation | dollars/month |
| $u_j, w_j$ | Relative allocation shares or normalized sample weights, as defined locally | dimensionless |
| $r, f, H, M$ | Savings rate, wants-to-needs ratio, habit and means components | dimensionless |
| $C_{k,t}$ | Cash in path $k$ on date $t$ | dollars |
| $\mu, \sigma$ | Annual return and volatility inputs in the implemented investment formula | annualized |
| $\Delta t$ | Elapsed investment time | years |
| $\widehat{\Delta}$ | After minus before mean weekly spending | dollars/week |

Symbols are local to their equations. In particular, $V$ denotes monthly investing in the budget equation and an investment value in the forecast update; the surrounding text and units distinguish them.

Appendix D

# Maintaining the manual

## D.1 Keep a task-based source

The editable book lives under `docs/manual/`. Its main TeX file controls chapter order; individual chapters contain the task procedures and methods. Instructional figures are original TikZ or PGFPlots sources. The illustrated cover is retained separately with its reference description and generation prompt in `figures/cover-doug-v6-provenance.md`. The generated tables and plot data come from fictional temporary databases. The PDF is bundled with the application and opens through **Help** > **User manual (PDF)**. It also remains independently readable.

The maintenance objective is a usable description of the current behavior. When a control changes, update its procedure, expected result, and relevant recovery steps together. When a calculation changes, update its explanation and worked evidence as well as its displayed label. A changed screenshot alone cannot establish that the described workflow still works.

## D.2 Use the source map

`source_map.json` connects chapters with their main implementation and verification files. It is an impact map, not a claim that the named tests were run while preparing this document. The chapter-end references let a maintainer locate the inspected behavior without requiring users to read source code to follow the task.

The generated manifest records SHA-256 hashes for the inspected Python package and selected supporting files. The generator copies application Python files to a temporary snapshot before importing them, then records the exact bytes used. The recorded commit is useful context, but cannot identify uncommitted changes. If files change during copying, the manifest identifies the individual copied bytes; it does not certify that the entire working tree was one atomic revision.

## D.3 Build and check an edition

From `docs/manual`, with the application's Python dependencies and a suitable TeX installation available, run:

```
make                  # examples, PDF, and document checks
make check-source     # compare current app files with the manifest
```

The example generator opens temporary fictional databases only. It publishes new generated evidence after its assertions finish and retains the diagnostic comparison even when that comparison identifies a model limitation. A failed assertion leaves the previous generated directory in place. Read a failure instead of deleting the assertion to obtain a successful build.

For editing prose without fresh application execution, use `make pdf`. The retained example date and

manifest then remain the calculation baseline. `make check` checks chapter structure and references, retained output hashes, generator identity, source-map paths, and the PDF build log. `make clean` removes build intermediates while retaining the PDF and evidence. The README provides interpreter selection and direct commands for environments without Make.

## D.4 Review what the checks cannot establish

1. Read every changed task against the actual interface and expected result, using fictional data when practical.
2. Check navigation labels, Simple and Everything visibility, both appearances (Dark and Light), dialog action names, signs, date conventions, and save-versus-draft behavior.
3. Inspect the cover, contents, tables, equations, figures, links, and page breaks in the generated PDF at readable scale.
4. Review diagnostic qualifications separately from pass assertions. Do not report a known discrepancy as an all-clear result.
5. Run the relevant application checks separately when needed. Document build checks do not certify the GUI or replace repository verification.
6. Run the source-drift check again after concurrent work settles. Review changed files through the source map and regenerate evidence only when the chapter descriptions are consistent with the new behavior.

A source-drift report means that the current bytes differ from the recorded baseline. It does not automatically diagnose a documentation error, and a clean report does not establish that every sentence is correct. New or removed Python files are reported as well as changed ones.

## D.5 Record revisions and preserve reproducibility

Change `ManualRevision` in the main TeX file and add a dated entry to the front-matter revision table. Record substantive behavior or scope changes in terms a user can recognize. Keep the application version separate from the manual revision. If the preparation date changes, update the cover date explicitly rather than allowing a PDF-only rebuild to imply a new review.

Retain `generated/provenance.json`, `generated/results.json`, the data and table files, and the corresponding document sources with an edition. The manifest records Python and dependency versions, generation date, generator hash, application source hashes, and output hashes. A seed alone is insufficient to reproduce a run after its inputs, date, implementation, or numerical dependencies change.

The first edition uses stable diagrams instead of screenshots because interface review was active. If screenshots are added later, use fictional records, identify the relevant edition, and crop them to the task being explained. Avoid including financial data, private paths, or writing-style reference documents in distributed artifacts.

## D.6 Bundle the matching edition

The Help action resolves `docs/manual/financing_with_doug_manual.pdf` through the application's shared resource lookup. This supports both a source checkout and a frozen installation. The PDF opens in the system's reader; a missing file or a failed launch produces a visible explanation.

The packaging specification includes the PDF through its `docs/` data files and stops if it is missing. Rebuild the document before packaging a revised edition. TeX sources and a Python example run are not required when a user opens the retained PDF. Its title page identifies the application version and manual revision, which should be reviewed together when preparing a release.

A successful local PDF or application build does not publish a release or update an installed copy. Ship the rebuilt PDF with the corresponding application package so that an installed user receives the intended edition.

Keep the manual readable independently of the program. Users should be able to find a task, understand its expected result, and investigate an unexpected amount without first learning implementation details. The calculation and maintenance parts provide the deeper explanation when it is needed.

# Index

account assignment, 19
accounts, 10
allocation policy, 46
appearance, 6

backup, 34
balance anchor, 10
balance reconstruction, 43
bootstrap, 49
bootstrap interval, 52
business reimbursement, 16

calendar, 19
categories, 16
checkpoints, 32
coverage, 52
CSV format, 54
CSV import, 13

demo data, 4
duplicate transactions, 13

earmarks, 23

fixed-decimal entry, 6
forecast, 28

goal progress, 46
goals, 23

history selection, 28
Home, 21

installation, 4
investment model, 49

keyboard shortcuts, 54

light theme, 6

manual maintenance, 61
merchant review, 16
month in review, 32
monthly allowance, 26
monthly budget, 26

navigation, 6

overrides, 26

parameter uncertainty, 49
pending transactions, 13
percentile bands, 28
permutation test, 52
posting date, 13
priority ladder, 23
purpose, 2

recency weighting, 43
reconciliation, 10
recurring patterns, 19
restore, 34
rollover, 46

Simple view, 6
source baseline, 61
spending score, 21, 43
Student-t innovations, 49

Track record, 32
transfers, 16

Undo, 34
unlocated savings, 23
updates, 34

verification, 52

weighted monthly average, 21
what-if events, 28
worked examples, 37